



Universidad
Internacional
de Valencia

Máster Universitario en Robótica y Automatización de Procesos Industriales

Sistemas Robóticos Móviles

Actividad 2

Simulación de celda warehouse con Pioneer, mannequin Bill, herramientas y SLAM en CoppeliaSim y Lua

Entrega práctica de Warehouse R1-B1-T1 y T2

Estudiante

Jorge Luis Mayorga Taborda

Profesor Jose I. Iñiguez

jiniguez@professor.universidadviu.com

Fecha 26 de mayo de 2026, antes de las 23:59

Índice general

Resumen	6
1 Introducción	7
1.1 Objetivos	7
1.2 Relación con el enunciado.....	8
1.3 Alcance de los algoritmos	8
1.4 Vista general del resultado	8
2 Escenario CoppeliaSim	10
2.1 Elementos del warehouse	10
2.2 Cómo funciona el Pioneer dentro de CoppeliaSim.....	11
2.3 Bill como actor móvil	11
2.4 Cola de tareas	11
2.5 Integración del código	12
2.6 Capturas de la simulación	14
2.7 Piezas visuales	14
3 Modelo del robot y control	15
3.1 Seguimiento de objetivo	15
3.2 Modos de control evaluados.....	16
4 Seguimiento de Bill y comparación de controladores	18
4.1 Geometría de seguimiento	19
4.2 Modelo cinemático usado por todos los modos	19
4.3 P, PI y PID	20
4.4 LQR discreto.....	21
4.5 NMPC por disparo directo	22
4.6 Resultados de seguimiento	23
5 Campos potenciales, evitación de obstáculos y planificación local	26
5.1 Campo atractivo hacia Bill	26
5.2 Campo reactivo de obstáculos	26
5.3 Visualización de sensores.....	27
5.4 Punto intermedio de planificación SLAM.....	27
6 Estimación y SLAM	28

6.1	Sensores: LiDAR, ultrasonidos y rayos de proximidad	28
6.2	Datos usados para comparar SLAM	29
6.3	Por qué hace falta Kalman	30
6.4	Predicción de pose	30
6.5	Corrección Kalman-landmark.....	31
6.6	GMapping en grid	32
6.7	Hector con ajuste local	32
6.8	Cartographer con submapas.....	33
6.9	Qué método usar y por qué	34
7	Resultados experimentales	35
7.1	Validación funcional de la escena	35
7.2	Desempeño del estimador Kalman-landmark	36
7.3	Comparación de algoritmos SLAM.....	37
7.4	Interpretación de resultados	38
8	Phase 2: SLAM con mapa desconocido	39
8.1	Recuperación ante bloqueo local	40
8.2	Comparación de métodos	41
8.3	Discusión técnica.....	43
9	Validación	44
9.1	Validación por lista de verificación	44
9.2	Timeline de la misión	44
9.3	Incidencias corregidas.....	46
	Conclusiones	47
	Referencias	49
A	Anexos	51
A.1	Archivos principales	51
A.2	Parámetros relevantes	53
A.3	Fragmentos Lua propios	53
A.4	Comparación de control	54
A.5	Parámetros de Cartographer	55
A.6	Notas de alcance	55

Índice de figuras

1.1	Vista superior de la celda base Sim_T2_Phase1_Basic.....	9
2.1	Máquina de estados resumida del controlador R1 para entrega, trabajo de Bill, retorno de herramienta y carga.....	12
2.2	Capturas 3D de la escena ejecutada en CoppeliaSim.	14
3.1	Comparación de modos de control exportada desde CoppeliaSim. Todos completan la tarea; PI obtiene el mejor puntaje ponderado con los pesos definidos.	17
4.1	Escena de seguimiento: Bill define un objetivo móvil y R1 regula distancia, rumbo y suavidad de ruedas.....	18
4.2	Capturas de ejecución de Actividad2_1_Basic_Follower.ttt: Bill recorre una trayectoria circular y R1 mantiene una distancia de seguimiento estable.	18
4.3	Referencia móvil del seguidor: el robot controla un punto detrás de Bill y no el centro de la persona.	19
4.4	Comparación temporal de P, PI, PID, LQR y NMPC en la escena de seguimiento de Bill.	23
4.5	Coste físico aproximado del seguidor: potencia instantánea y energía acumulada en las ruedas.	24
4.6	Suavidad de comandos del seguidor: variación de las órdenes de rueda y cambios de sentido.....	25
6.1	Secuencia de estimación, mapa, planificación local y control de R1.....	29
6.2	Particularidad de Kalman-landmark: corrige la pose y fusiona pocas detecciones como rasgos; no mantiene una covarianza conjunta pose–mapa como un EKF-SLAM completo.	31
6.3	Particularidad de GMapping implementado: la información principal es una grid de ocupación. Cada rayo libera celdas antes del impacto y refuerza la celda ocupada.	32
6.4	Particularidad de Hector: busca la pose local cuyo scan encaja mejor con la grid. La odometría inicializa la búsqueda, pero la corrección viene del ajuste scan–mapa.	33
6.5	Particularidad de Cartographer-submap: el mapa no se guarda como una sola grid monolítica, sino como submapas locales con posibles cierres cuando R1 vuelve a zonas conocidas.	33
7.1	Resultados del estimador Kalman-landmark.....	37
7.2	Comparación gráfica de los cuatro modos SLAM: trayectoria, precisión, crecimiento del mapa y resumen de error.	38
8.1	Capturas 3D de la escena Phase 2 ejecutada en CoppeliaSim.	39

8.2	Capturas de ejecución de Phase 2 con mapa desconocido y obstáculos en la celda.	40
8.3	Comparación de métodos en Phase 2.	42
8.4	Reconstrucción del mapa al mismo instante común, $t = 60$ s.	42
9.1	Timeline exportado de la simulación: estados de tarea, movimiento, planificador y acciones de Bill durante la misión.	45
9.2	Panel de la ejecución base: pose, error, batería, obstáculos, planificador y estados principales.	45
A.1	Métricas complementarias de control: tiempos, trayectoria, suavidad, batería, riesgo y error de pose.	55

Índice de cuadros

1.1	Trazabilidad entre el enunciado de la Actividad 2 y la solución implementada.	8
2.1	Elementos de la celda de almacén simulada en CoppeliaSim.	10
4.1	Métricas del seguidor de Bill por modo de control.	23
6.1	Criterio de selección entre las variantes SLAM implementadas.	34
7.2	Comparación SLAM en CoppeliaSim.	37
8.1	Comparación Phase 2 con mapa desconocido y obstáculos dinámicos.	41
A.1	Escenas y controladores principales.	51
A.2	Exportadores, registros y escena complementaria.	52
A.3	Resultados comparativos de los cinco controladores de seguimiento en Fase 1.	54
A.4	Parámetros de Cartographer-submap. Idénticos en Fase 1 y Fase 2.	55

Resumen

Se implementó en CoppeliaSim y Lua un Pioneer P3DX dentro de una celda de almacén. La escena base `Sim_T2_Phase1_Basic.ttt` integra los elementos principales del escenario: R1, B1 (Bill) como persona móvil, T1 y T2, dos mesas, estanterías, un sofá de operador, obstáculos y la estación de carga. B1 recibe en el código los identificadores /B1 y Bill; corresponde al modelo de persona de CoppeliaSim instanciado como `People/mannequin`.

R1 lee los sensores de proximidad, estima su pose, actualiza un mapa local, evita obstáculos y comanda las ruedas diferenciales mientras coordina entregas con B1. El ciclo validado comprende las cuatro tareas siguientes.

- `task1:{T1 -> B1@WS1}`: R1 recoge T1 en la estantería y la entrega a B1.
- `task2:{B1@WS1(T1) -> Rack_T1}`: B1 trabaja la pieza y R1 la devuelve a su estantería.
- `task3:{T2 -> B1@WS2}`: B1 camina a la segunda mesa y solicita T2.
- `task4:{B1@WS2(T2) -> Rack_T2}`: R1 recoge la pieza terminada y la almacena.

La comparación de controladores (P, PI, PID, LQR y NMPC) se realizó en una escena separada de seguimiento (`Actividad2_1_Basic_Follower.ttt`), con PI como mejor balance en esa escena de seguimiento según el puntaje ponderado definido. La escena `warehouse` usa PID como modo base. El NMPC completo (horizonte $N=10$, $\Delta t = 0,22$ s) opera exclusivamente en el controlador del follower (`pioneer_pid_follower_controller.lua`).

La segunda escena validada es `Sim_T2_Phase2_SLAM_Unknown.ttt`. En esa escena, el robot empieza con parte del entorno oculta y se encuentra obstáculos que aparecen durante la misión. En esa fase se comparan cuatro variantes de estimación y mapeo simultáneo SLAM (*Simultaneous Localisation and Mapping*) escritas en Lua: `KALMAN_LANDMARK`, `GMAPPING_GRID`, `HECTOR_GRID_MATCHING` y `CARTOGRAPHER_SUBMAP`. Usan las mismas tareas, sensores y controlador PID, para que la comparación de representaciones de mapa no dependa de condiciones de escena distintas.

La ejecución final de Phase 2 completó las cuatro tareas, volvió a carga, mantuvo activos los 16 sensores. En el ensayo con mapa desconocido, Kalman-landmark quedó con el menor RMSE de posición (0,03185 m) y la menor duración (124,80 s). Cartographer-submap generó más actividad de mapa, con 13 submapas y 4 cierres de ciclo locales; Hector terminó con el mejor indicador aproximado de precisión entre las grids.

1

Introducción

Se implementó un Pioneer P3DX en CoppeliaSim y Lua capaz de navegar autónomamente en una celda robotizada con evitación de obstáculos, conforme a los requisitos de la Actividad 2 [1]. La escena representa un caso de fábrica sencillo: R1 asiste a B1, llamado Bill, una persona tomada del modelo disponible en CoppeliaSim y usada como *mannequin* operativo. Los fundamentos de robótica móvil diferencial y navegación autónoma sobre los que se construye el sistema están descritos en [2]. El robot lleva piezas o herramientas a dos mesas, espera mientras Bill trabaja, las recoge, las devuelve a su estantería y termina en carga.

El ciclo básico incluye coordinación temporal, evitación local, diferenciación entre estantería y bloqueo, retorno de herramienta y regreso a carga. La escena principal se guardó como:

Las cuatro variantes de estimación y mapeo implementadas (KALMAN_LANDMARK, GMAPPING_GRID, HECTOR_GRID_MATCHING y CARTOGRAPHER_SUBMAP) son implementaciones propias en Lua, inspiradas conceptualmente en los algoritmos del mismo nombre pero sin sus optimizadores originales: la variante de GMapping implementa una grid de ocupación log-odds sin filtro de partículas Rao-Blackwellizado; la de Hector realiza una búsqueda discreta de pose en lugar de ajuste Gauss-Newton multi-resolución; la de Cartographer aplica cierre de ciclo local en lugar de optimización global de grafo de poses. La comparación evalúa cuatro *representaciones de mapa* bajo las mismas condiciones de misión, no la equivalencia con los paquetes ROS originales.

```
actividad2/coppeliasim/Sim_T2_Phase1_Basic.ttt
```

1.1 Objetivos

- Implementar un Pioneer P3DX diferencial en CoppeliaSim y Lua que navegue de forma autónoma en una celda de almacén.
- Seguir al *mannequin* Bill con control de distancia y rumbo en la escena de seguimiento, comparando los modos P, PI, PID, LQR (*Linear Quadratic Regulator*) y NMPC (*Nonlinear Model Predictive Control*) (escena Actividad2_1_Basic_Follower.ttt); la escena warehouse usa PID como controlador base.
- Planificar y ejecutar las entregas de T1 y T2 con una máquina de estados reproducible y validable.
- Evitar obstáculos en tiempo real con campos potenciales reactivos y recuperación ante bloqueos locales.
- Comparar cuatro variantes de estimación y mapeo simultáneo SLAM (*Simultaneous Localisation and Mapping*) en Lua —Kalman-landmark, grid log-odds, ajuste por scan-matching y submapas—

con métricas exportadas.

1.2 Relación con el enunciado

La guía pide programar un Pioneer P3DX en CoppeliaSim/Lua, primero para avanzar hacia un *mannequin* mediante campos potenciales y después para esquivar obstáculos mientras mantiene ese comportamiento. La entrega también exige la integración en una celda robotizada, un PDF y una presentación.

En este trabajo el *mannequin* se implementa como Bill/B1, el modelo de persona móvil disponible en CoppeliaSim. R1, el Pioneer, no solo se acerca a Bill: sigue una referencia detrás de la persona, transporta T1 y T2, respeta esperas de trabajo humano, evita pallet, pilar, caja y obstáculos de Phase 2, y vuelve a la estación de carga. Se añade una capa de validación cuantitativa con métricas exportadas, comparación de controladores y mapeo local (Tabla 1.1).

Punto de la guía	Dónde queda cubierto
Seguir al <i>mannequin</i>	Bill/B1 actúa como objetivo móvil y como solicitante de herramientas en la celda.
Campos potenciales	Capítulo de campos potenciales: atracción hacia Bill y repulsión por sensores.
Anticolisión	Lectura de 16 sensores de proximidad, reducción de velocidad, giro reactivo y parada crítica.
Integración en celda	Warehouse con racks, mesas WS1/WS2, herramientas T1/T2, estación de carga y máquina de estados.
Ampliación del script base	Seguimiento P/PI/PID/LQR/NMPC, estimación Kalman, variantes SLAM y Phase 2 con mapa desconocido.

Cuadro 1.1: Trazabilidad entre el enunciado de la Actividad 2 y la solución implementada.

1.3 Alcance de los algoritmos

Los algoritmos corren dentro de CoppeliaSim y Lua. Su alcance es acotado respecto a paquetes ROS completos: Kalman-landmark combina odometría, corrección de pose con mediciones ruidosas y rasgos compactos de obstáculo. Las variantes inspiradas en GMapping, Hector y Cartographer implementan respectivamente: mapeo por grid log-odds, corrección de pose por scan-matching discreto y gestión de submapas con cierre de ciclo local. Ninguna replica el estimador probabilístico original (filtro de partículas, Gauss-Newton o grafo de poses), pero mantiene la representación de mapa característica de cada enfoque.

1.4 Vista general del resultado

La Fig. 1.1 muestra la vista superior de la celda base, con R1, B1, las estanterías de T1 y T2, las mesas de trabajo y los obstáculos presentes desde el inicio.

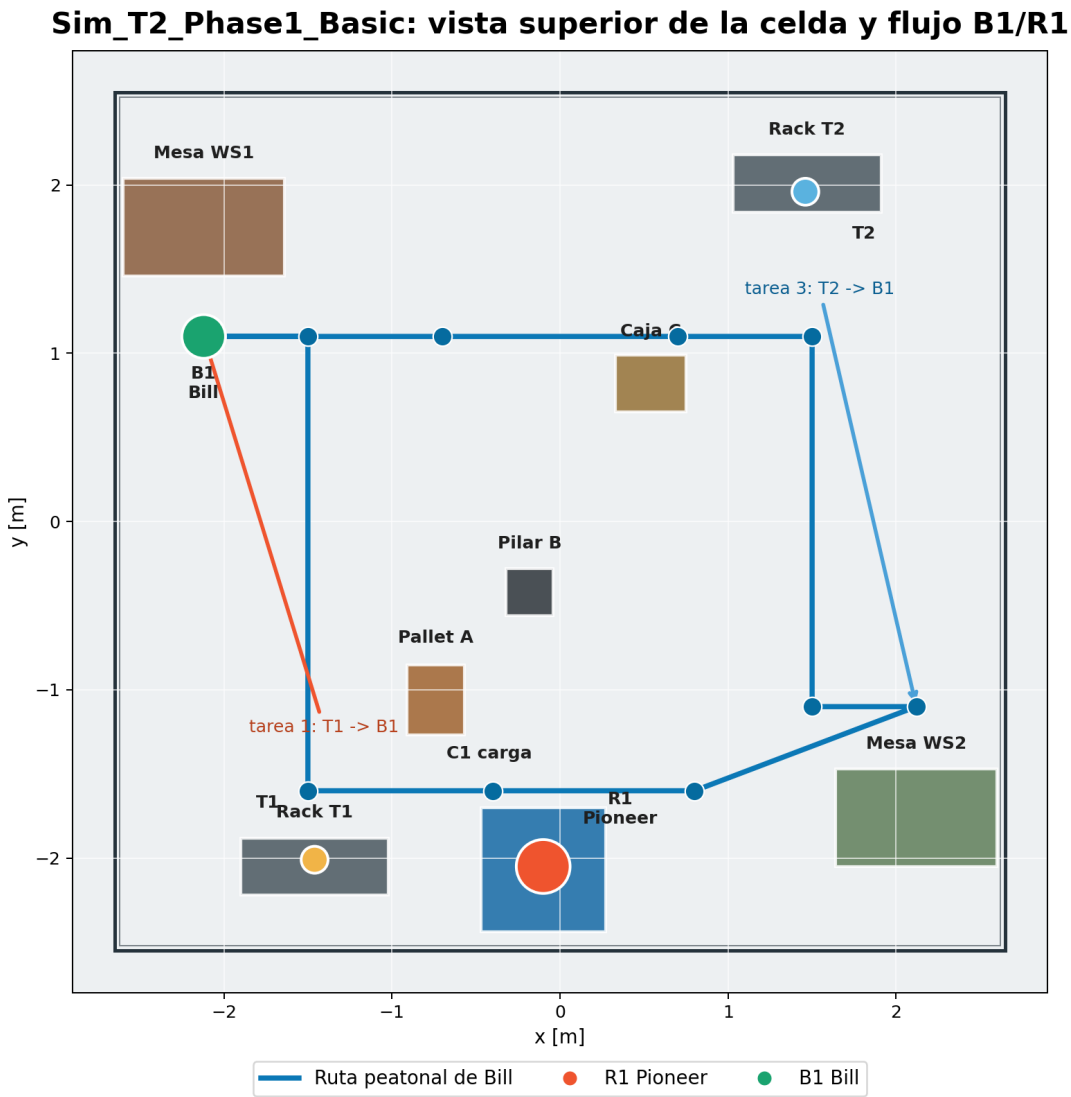


Figura 1.1: Vista superior de la celda base Sim_T2_Phase1_Basic.

2

Escenario Coppeliasim

La escena se genera con `actividad2/coppeliiasim/build_phase1_basic_scene.py`. Ese script carga la base de la asignatura, limpia restos de pruebas anteriores, coloca la celda, ubica a R1 y B1, añade estanterías, mesas, sofá, obstáculos y piezas visuales, y deja integrados los controladores Lua [3]. Al final lanza una validación sin interfaz gráfica y guarda un JSON reproducible con estado, métricas y objetos verificados.

2.1 Elementos del warehouse

La Tabla 2.1 lista los elementos principales de la celda de almacén.

Elemento	Alias en escena	Función
Robot móvil	/PioneerP3DX	R1 ejecuta la cola de tareas, navega, evita obstáculos, transporta herramientas y vuelve a carga.
Persona móvil	/B1, Bill	Modelo de persona de Coppeliasim renombrado para actuar como People/mannequin; trabaja en dos mesas, camina entre WS1 y WS2 y solicita herramientas.
Herramienta/pieza 1	/T1	Pieza mecanizada visual, entregada en WS1 y devuelta a Rack_T1.
Herramienta/pieza 2	/T2	Pieza tipo placa electrónica/compuesta, entregada en WS2 y devuelta a Rack_T2.
Estación de carga	/C1	Punto final de retorno y recarga del robot.
Mesas de trabajo	/P1_WorkTable_1, /P1_WorkTable_2	Superficies donde Bill toma la pieza, la trabaja y la devuelve a R1.
Sofá de operador	/P1_Operator_Sofa_*	Elemento de descanso y referencia espacial dentro de la celda, ubicado fuera de la ruta principal.
Estanterías	/Rack_T1, /Rack_T2	Estaciones de almacenamiento opuestas para T1 y T2.
Obstáculos	Pallet, pilar y caja	Elementos detectables para activar evitación reactiva y mapeo.

Cuadro 2.1: Elementos de la celda de almacén simulada en Coppeliasim.

2.2 Cómo funciona el Pioneer dentro de Coppeliasim

El Pioneer P3DX de la escena se maneja como una plataforma diferencial. Tiene dos ruedas motrices laterales: si las dos ruedas giran a la misma velocidad, el robot avanza recto; si una rueda gira más rápido que la otra, el robot cambia de rumbo; si una avanza y la otra retrocede, puede girar casi sobre su propio centro. Por eso el controlador no envía una posición final directamente al robot. En cada ciclo calcula una velocidad lineal v y una velocidad angular ω , las convierte en velocidades de rueda y las aplica a los motores de Coppeliasim.

El ciclo de control es siempre el mismo:

1. Leer la pose estimada de R1, el estado de Bill, el objetivo de la tarea y los sensores de proximidad.
2. Actualizar el estimador y el mapa local con las detecciones del ciclo.
3. Decidir si el objetivo directo es seguro o si conviene pasar por un SLAM_WAYPOINT.
4. Calcular v y ω con atracción al objetivo, corrección de rumbo y repulsión de obstáculos.
5. Convertir v, ω a rueda izquierda y derecha, saturar los mandos y registrar telemetría.

Este bucle explica por qué el documento separa escena, cinemática, campos potenciales, anticolisión y SLAM. No son módulos aislados: todos alimentan la misma decisión de velocidad que se envía al Pioneer.

2.3 Bill como actor móvil

Bill camina entre las dos estaciones, pide herramientas y trabaja la pieza en la mesa que toca. El script `phase1_b1_walking_manager.lua` define WS1/WS2, escribe la estación actual, indica qué herramienta necesita y anima las acciones de tomar, trabajar y entregar. La orientación se calcula con:

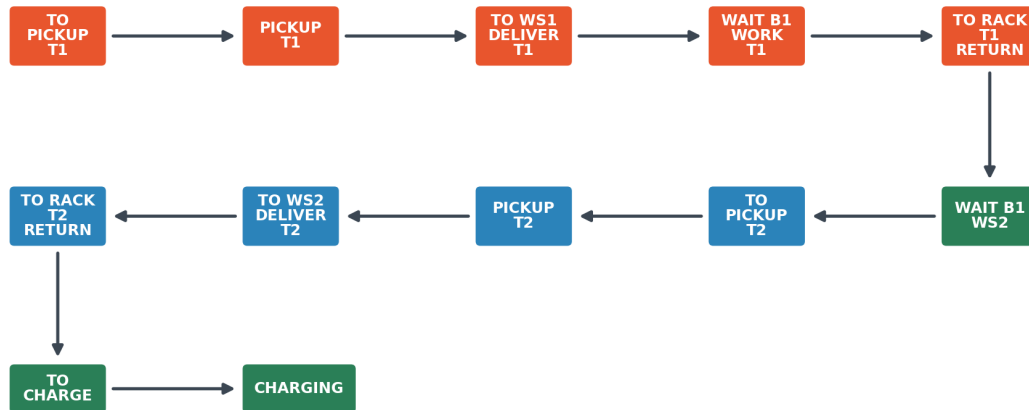
$$\theta_B = \text{atan2}(y_{k+1} - y_k, x_{k+1} - x_k)$$

Bill mira hacia donde camina. En las mesas se orienta hacia la superficie de trabajo para mantener coherencia visual entre la animación y la tarea ejecutada.

2.4 Cola de tareas

La cola de tareas se mueve con señales internas de Coppeliasim y una máquina de estados dentro del controlador de R1. La Figura 2.1 resume la máquina de estados. R1 espera a que Bill termine T1 y llegue a WS2 antes de buscar T2; esta sincronización evita que la segunda entrega empiece antes de una solicitud válida.

Maquina de estados resumida del controlador R1



Cada flecha entra y sale de un estado: llegada por tolerancia, manipulación por temporizador y sincronización por señales de Bill.

Figura 2.1: Máquina de estados resumida del controlador R1 para entrega, trabajo de Bill, retorno de herramienta y carga.

2.5 Integración del código

El código de la escena se concentra en percepción, coordinación de tareas y registro de datos. El mannequin queda como persona móvil /B1; los sensores entregan datos a la evitación y al mapa; B1 tiene su propio manager, con tiempos de trabajo y movimiento; y los exportadores guardan CSV y JSON para revisar control, seguridad y SLAM con datos reproducibles.

Listing 2.1: Lectura de sensores y evitación reactiva en Lua.

```

for _, s in ipairs(sensors) do
    local ok, distance, point, object = sim.readProximitySensor(s.handle)
    if ok and ok > 0 and distance > 0 then
        addSensorRay(s, distance, point, true)
        if object ~= b1 then
            registerSlamDetection(s, distance, point)
        end
        if distance < cfg.avoidRange then
            local influence = ((cfg.avoidRange - distance) / cfg.avoidRange)^2
            steer = steer - sign(s.y) * cfg.kRepulsion * influence
            slow = math.max(slow, influence)
        end
    else
        addSensorRay(s, cfg.sensorVizRange, nil, false)
    end
end
end
  
```

Listing 2.2: Fragmento de la máquina de estados de T1 y T2.

```

local TaskHandlers = {
  
```

```

PICKUP_T1 = function()
    pickupTool(t1, 'T1', 'TO_WS1_DELIVER_T1')
end,
DELIVER_T1_WS1 = function()
    deliverToolToBill(t1, ws1WorkSurface, 1,
        'task1:{T1->B1@WS1}', 'WAIT_B1_WORK_T1')
end,
WAIT_B1_WS2_REQUEST_T2 = function()
    stopRobot('WAIT_B1')
    if readStringSignal('phase1B1Station', 'NONE') == 'WS2' then
        setTaskState('TO_PICKUP_T2')
    end
end,
}

```

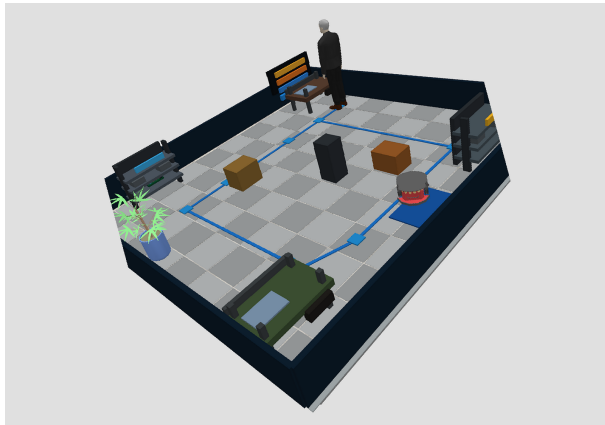
Listing 2.3: Selección de objetivo directo o punto intermedio SLAM.

```

local targetWorld = readWorldPosition(target)
local goalX, goalY = targetWorld[1], targetWorld[2]
local planX, planY = plannedWaypointTo(goalX, goalY, tolerance)
local dx, dy = planX - estX, planY - estY
local heading = normalizeAngle(atan2(dy, dx) - estTheta)
local obstacleSteer, obstacleSlow, minObstacle, hardStop = readObstacleField()
local v, w = computeCommand(distance, heading, lateral,
    obstacleSteer, obstacleSlow, hardStop)
setWheelSpeeds(v, w)

```

2.6 Capturas de la simulación



(a) Escena completa en CoppeliaSim.



(b) Rayos de sensor y evitación local.



(c) Bill trabajando frente a la mesa.



(d) Entrega de herramienta entre R1 y Bill.

Figura 2.2: Capturas 3D de la escena ejecutada en CoppeliaSim.

2.7 Piezas visuales

T1 y T2 se hicieron con primitivas para evitar geometrías genéricas. T1 lleva placa, nervios, boss central y pernos; T2 representa una placa compuesta/electrónica con raíles, conector y capacitor. La geometría diferenciada distingue cada herramienta durante la ejecución y en las capturas.

3

Modelo del robot y control

El Pioneer P3DX se trata como robot diferencial [4]. El controlador calcula una velocidad lineal v y una velocidad angular ω , y de ahí salen las velocidades de rueda izquierda y derecha:

$$v_L = v - \frac{L}{2}\omega, \quad v_R = v + \frac{L}{2}\omega$$

Se adopta la convención usual de CoppeliaSim para esta escena: $\omega > 0$ incrementa el yaw del robot en sentido antihorario visto desde arriba. Con esa convención, la rueda derecha recibe mayor velocidad que la izquierda durante un giro positivo.

Si r es el radio de rueda, las consignas enviadas a los motores de CoppeliaSim son:

$$\omega_L = \frac{v_L}{r}, \quad \omega_R = \frac{v_R}{r}$$

En la implementación se usan $r = 0,0975$ m y $L = 0,331$ m, valores compatibles con el Pioneer P3-DX [5]. Las velocidades se saturan para mantener los comandos dentro de un rango físico y conservar la repetibilidad de las ejecuciones.

3.1 Seguimiento de objetivo

Para navegar hacia un rack, mesa o cargador, se calcula primero la posición del objetivo en coordenadas globales y luego el error respecto a la pose estimada del robot:

$$e_x = x_g - \hat{x}, \quad e_y = y_g - \hat{y}, \quad d = \sqrt{e_x^2 + e_y^2}$$

El rumbo deseado es:

$$\theta_g = \text{atan2}(e_y, e_x)$$

y el error angular usado por el controlador es:

$$e_\theta = \text{wrap}(\theta_g - \hat{\theta})$$

La consigna base se corrige con un término lateral y con una contribución de evitación de obstáculos:

$$v = f_d(d - d_f) \cos(e_\theta), \quad \omega = f_\theta(e_\theta) + f_{lat}(e_y^{body}) + f_{obs}$$

donde d_f es la distancia de seguimiento y f_{obs} resume el giro repulsivo derivado de sensores.

3.2 Modos de control evaluados

Se probaron cinco modos: P, PI, PID, LQR y NMPC. Los controladores P, PI y PID se usan como referencia clásica junto con la estimación y la evitación. LQR y NMPC se incluyeron con alcance acotado: LQR usa una ganancia discreta calculada fuera de CoppeliaSim mediante Riccati; NMPC usa disparo directo con búsqueda local de comandos.

Modo	Idea	Uso en la escena
P	Control proporcional sobre distancia y rumbo.	Estructura simple y estable, con mayor tiempo activo y ruta más larga.
PI	Agrega integral de distancia y rumbo.	Obtiene el menor puntaje ponderado en la comparación de controladores.
PID	Incluye derivadas para amortiguar saltos.	Buen compromiso general; se usó como modo base en varias exportaciones SLAM.
LQR	Realimentación lineal de error longitudinal, lateral y rumbo.	Ganancia discreta calculada por Riccati e integrada en Lua para el seguidor de Bill.
NMPC	Control predictivo no lineal por disparo directo.	Evalúa secuencias de v, ω con restricciones y coste de suavidad mediante búsqueda local (véase Sección 4.5).

El puntaje ponderado que determina el modo ganador se calcula como la suma min-max normalizada de seis indicadores, con los pesos:

$$S = 0,28 s_{\text{mov}} + 0,18 s_{\text{bat}} + 0,20 s_{\text{var}} + 0,14 s_{\text{riesgo}} + 0,12 s_{\text{pose}} + 0,08 s_{\text{dur}}$$

donde menor puntaje es mejor. Los pesos reflejan las prioridades de una tarea de seguimiento suave y eficiente: mayor importancia al tiempo en movimiento activo (0,28) y a la suavidad de comandos (0,20), seguidos del consumo energético (0,18) y el riesgo de obstáculo (0,14), con menor peso al error de pose (0,12) y la duración total (0,08). Los pesos son ad hoc y calibrados para esta escena; la ordenación de controladores es robusta ante variaciones moderadas ($\pm 20\%$) del peso principal. Los valores coinciden con los definidos en `run_phase1_controller_comparison.py` (véase también el Anexo A.3). La Fig. 3.1 resume los resultados de los cinco modos.

Phase 1 Basic - Comparacion de controladores en la misma secuencia T1/T2

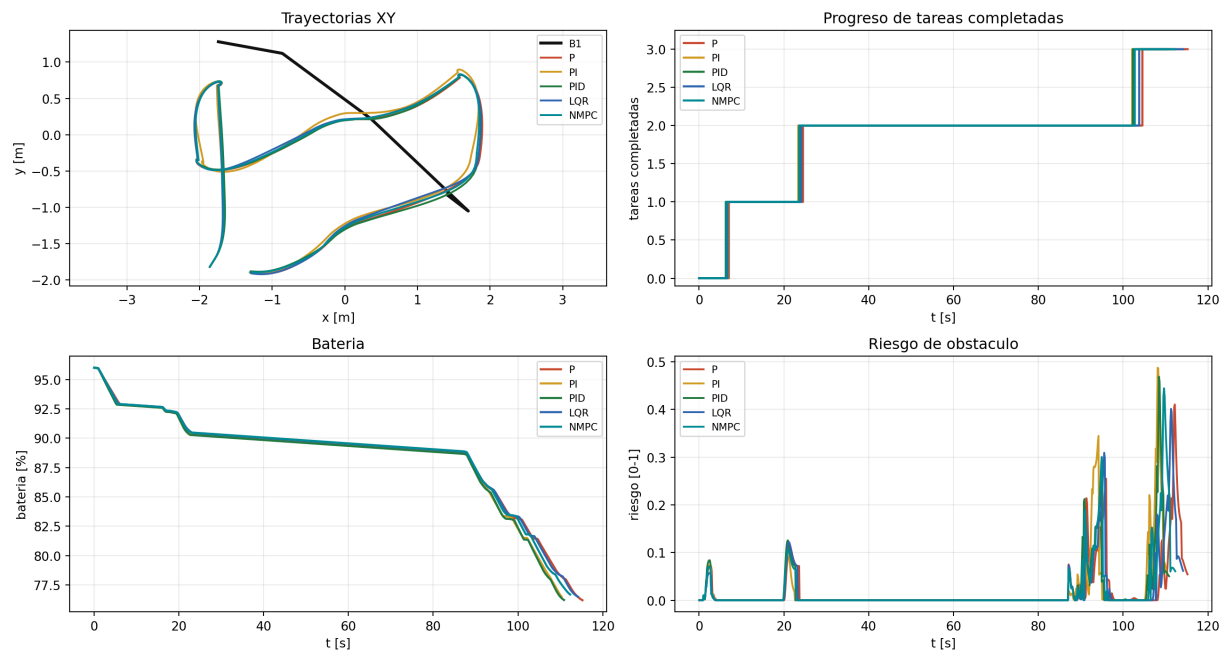


Figura 3.1: Comparación de modos de control exportada desde CoppeliaSim. Todos completan la tarea; PI obtiene el mejor puntaje ponderado con los pesos definidos.

4

Seguimiento de Bill y comparación de controladores

Antes de integrarlo en la celda completa, se probó el seguimiento de Bill en `Actividad2_1_Basic_Follower.ttt` (Fig. 4.1, capturas en Fig. 4.2). Bill camina con aceleración y frenado, y el Pioneer sigue un punto detrás de la persona. Si se apunta al centro del cuerpo, el robot tiende a acercarse demasiado; por eso la referencia va detrás de Bill.

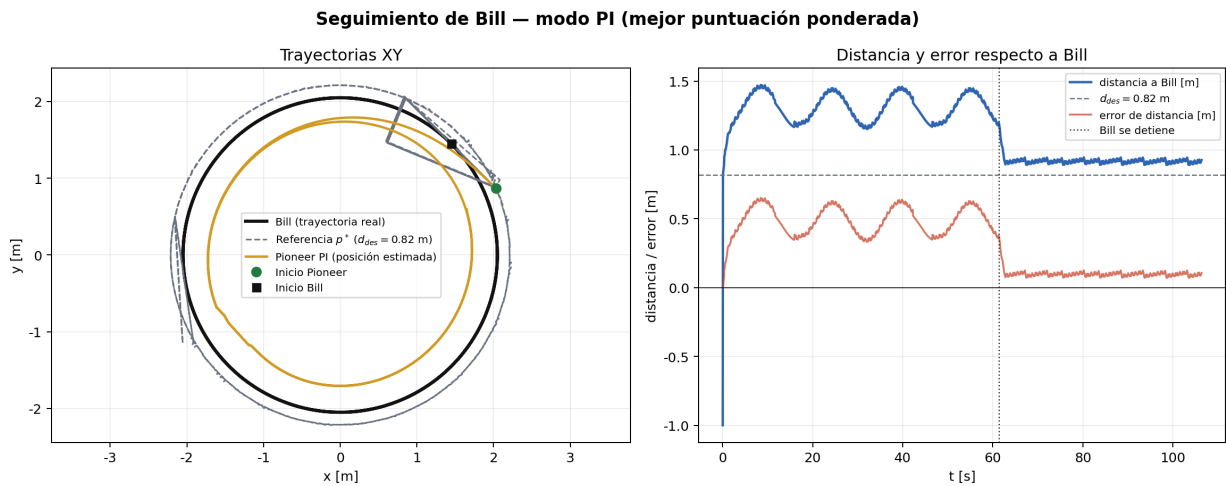


Figura 4.1: Escena de seguimiento: Bill define un objetivo móvil y R1 regula distancia, rumbo y suavidad de ruedas.

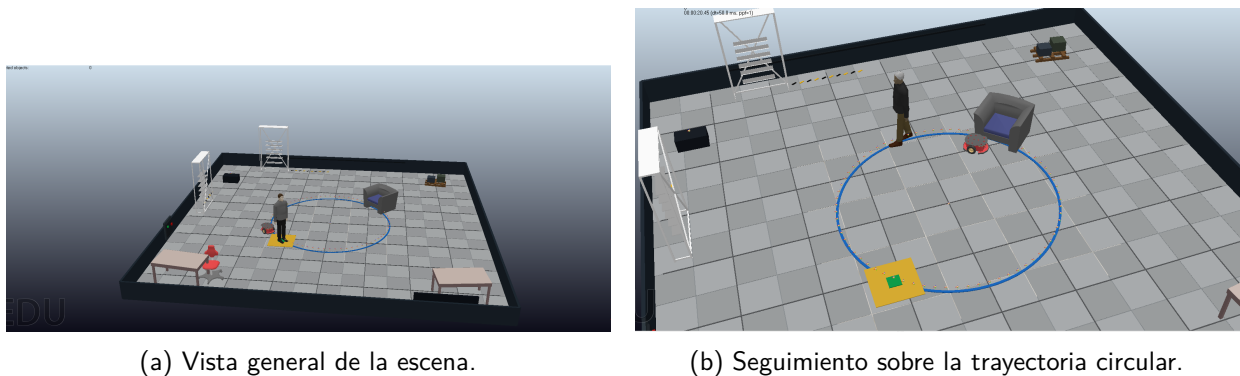


Figura 4.2: Capturas de ejecución de `Actividad2_1_Basic_Follower.ttt`: Bill recorre una trayectoria circular y R1 mantiene una distancia de seguimiento estable.

4.1 Geometría de seguimiento

Si $p_B = [x_B, y_B]^T$ es la posición de Bill, θ_B su orientación y d_{des} la separación deseada, el punto de referencia se calcula como:

$$p^* = p_B - d_{des} \begin{bmatrix} \cos \theta_B \\ \sin \theta_B \end{bmatrix}, \quad d_{des} = 0,82 \text{ m}.$$

El error de distancia se mide con respecto a Bill, mientras que el error de rumbo se calcula desde el robot hacia Bill (Fig. 4.3):

$$e_d = \|p_B - p_R\| - d_{des}, \quad e_\psi = \text{wrap}(\text{atan2}(y_B - y_R, x_B - x_R) - \theta_R).$$

Parte 1 follower: geometria de seguimiento Bill-Pioneer

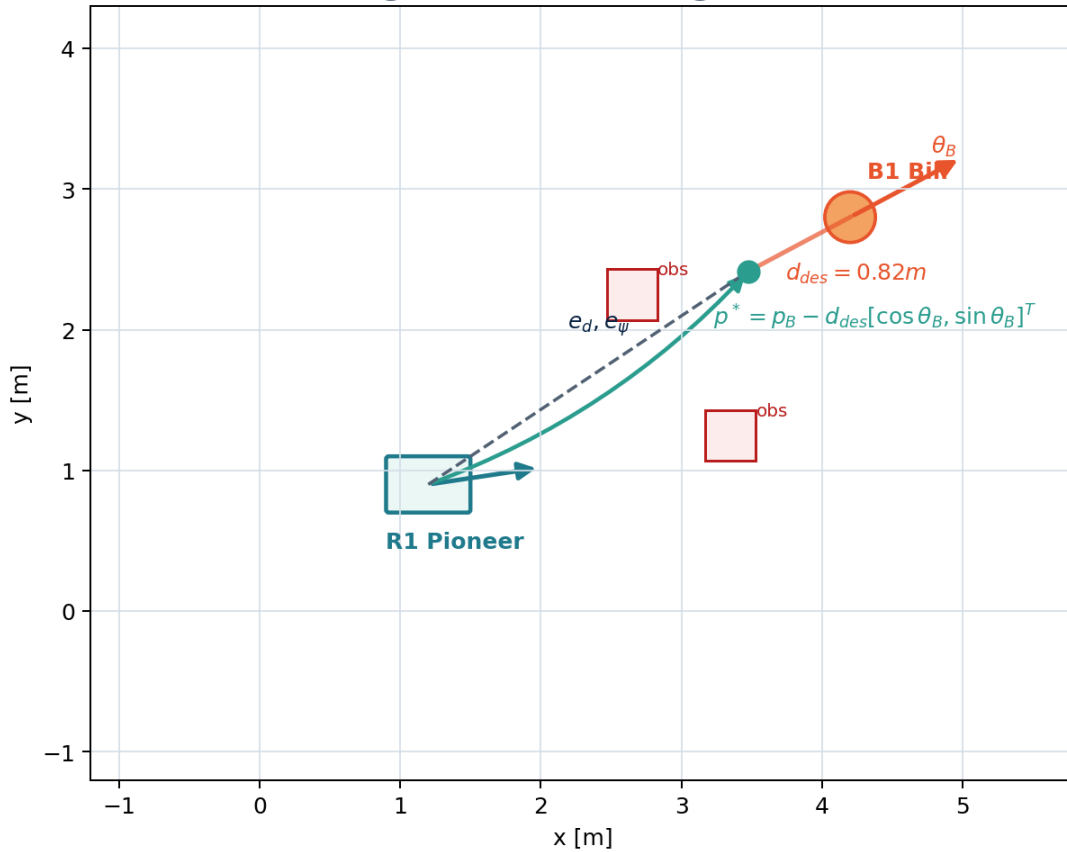


Figura 4.3: Referencia móvil del seguidor: el robot controla un punto detrás de Bill y no el centro de la persona.

4.2 Modelo cinemático usado por todos los modos

El movimiento plano del Pioneer se aproxima con el modelo unicycle:

$$\dot{x} = v \cos \theta, \quad \dot{y} = v \sin \theta, \quad \dot{\theta} = \omega.$$

La conversión a velocidades angulares de rueda usa el radio r y la distancia entre ruedas L :

$$\omega_L = \frac{v - \frac{L}{2}\omega}{r}, \quad \omega_R = \frac{v + \frac{L}{2}\omega}{r}, \quad r = 0,0975 \text{ m}, \quad L = 0,331 \text{ m}.$$

Estas mismas señales se usan para calcular potencia aproximada, energía, variación de comando y suavidad de control.

La energía de la Tabla 4.1 se calcula como una métrica mecánica aproximada. Se obtiene en Lua con la potencia de avance de cada rueda, fricción de rodadura, un término viscoso de rueda y potencia de guiñada:

$$\begin{aligned} P_L &= \left| \frac{m}{2} a v_L \right| + \frac{F_r}{2} |v_L| + b_\omega \omega_L^2, \\ P_R &= \left| \frac{m}{2} a v_R \right| + \frac{F_r}{2} |v_R| + b_\omega \omega_R^2, \\ P_\theta &= |I_z \dot{\omega}|, \quad E = \sum_k (P_L + P_R + P_\theta) \Delta t. \end{aligned}$$

Los parámetros usados fueron $m = 16,5 \text{ kg}$, $I_z = 0,72 \text{ kg m}^2$, $F_r = 2,8 \text{ N}$ y $b_\omega = 0,006 \text{ W rad}^{-2} \text{ s}^2$. En esa expresión, $a = \dot{v}$ es la aceleración lineal estimada entre muestras y $\dot{\omega}$ es la aceleración angular. Esta magnitud se usa como indicador relativo entre controladores dentro de la misma escena; no representa una estimación eléctrica de batería.

Los términos $\frac{m}{2} a v_L$ y $\frac{m}{2} a v_R$ reparten aproximadamente la potencia inercial lineal, no como un modelo dinámico completo de rueda. La inercia real actúa sobre el centro de masa y se transmite por los contactos; se reparte por rueda solo para obtener una métrica relativa sensible a frenazos, inversiones de sentido y oscilaciones de comando.

4.3 P, PI y PID

Los controladores clásicos se aplican sobre e_d y se combinan con una realimentación de rumbo:

$$\begin{aligned} u_P &= K_p e_d, \quad u_{PI} = K_p e_d + K_i \sum_{k=0}^n e_d(k) \Delta t, \\ u_{PID} &= K_p e_d + K_i \sum_{k=0}^n e_d(k) \Delta t + K_d \frac{e_d(k) - e_d(k-1)}{\Delta t}. \end{aligned}$$

El término proporcional responde al error instantáneo, el integral corrige error estacionario cuando Bill mantiene movimiento, y el derivativo amortigua saltos bruscos. En ejecución, todos los modos añaden una corrección de evitación de obstáculos al ángulo de giro:

$$\omega = \omega_{\text{controlador}} + \omega_{\text{obs}},$$

donde ω_{obs} es el término repulsivo de la sección de campos potenciales (§5). Todos los modos pasan por saturaciones de velocidad y aceleración.

4.4 LQR discreto

El modo LQR del seguidor regula el error local alrededor del punto p^* , siguiendo la forma clásica de control cuadrático lineal [6]. El estado se define en el marco del robot mediante la transformación:

$$\begin{aligned} e_{long} &= \cos \theta_R (x_{p^*} - x_R) + \sin \theta_R (y_{p^*} - y_R), \\ e_{lat} &= -\sin \theta_R (x_{p^*} - x_R) + \cos \theta_R (y_{p^*} - y_R), \quad e_{yaw} = \text{wrap}(\psi_{ref} - \theta_R), \end{aligned}$$

donde (x_{p^*}, y_{p^*}) es el punto de referencia p^* . El vector de estado y la acción de control son:

$$x_L = [e_{long} \quad e_{lat} \quad e_{yaw}]^T, \quad u = [\Delta v \quad \Delta \omega]^T.$$

Con $v_{ref} = 0,22$ m/s, radio local de giro $\rho_{ref} = 1,55$ m, $\omega_{ref} = v_{ref}/\rho_{ref}$ y $\Delta t = 0,05$ s, la linealización usada en `compute_follower_lqr_gain.py` es:

$$A = \begin{bmatrix} 0 & \omega_{ref} & 0 \\ -\omega_{ref} & 0 & v_{ref} \\ 0 & 0 & 0 \end{bmatrix}, \quad B = \begin{bmatrix} -1 & 0 \\ 0 & 0 \\ 0 & -1 \end{bmatrix}.$$

$$A_d = I + \Delta t A, \quad B_d = \Delta t B.$$

El script resuelve la ecuación algebraica de Riccati discreta con:

$$Q = \text{diag}(35, 80, 18), \quad R = \text{diag}(6, 3),$$

$$K = (B_d^T P B_d + R)^{-1} B_d^T P A_d.$$

Los pesos proceden de una calibración empírica sobre el seguimiento de Bill, no de una identificación formal del Pioneer. Se penalizó más el error lateral ($q_{lat} = 80$) que el longitudinal ($q_{long} = 35$), porque el robot admite permitirse ir algo adelantado o retrasado, pero no cruzarse lateralmente con la referencia móvil. El coste de giro en R se dejó menor que el de avance para que el regulador prefiriera corregir rumbo antes de cambiar mucho la velocidad.

La ley aplicada en Lua usa $u = -Kx_L$, con saturación posterior de v y ω .

La matriz integrada en Lua fue:

$$K = \begin{bmatrix} -2.3496 & 1.2925 & 0.1203 \\ 0.2427 & -4.3570 & -2.6887 \end{bmatrix}.$$

Esta parte corresponde a un LQR discreto real, de orden bajo y aplicado al seguimiento local de Bill.

4.5 NMPC por disparo directo

El modo NMPC se implementa como control predictivo no lineal por disparo directo. Mantiene una secuencia de v_k, ω_k , simula el modelo hacia adelante y mejora esa secuencia con dos pasadas de búsqueda local, siguiendo la estructura general de MPC [7]. La optimización se resuelve sobre la secuencia de comandos; IPOPT y CasADi no se usan en esta implementación.

$$x_{k+1} = x_k + v_k \cos(\theta_k) \Delta t, \quad y_{k+1} = y_k + v_k \sin(\theta_k) \Delta t,$$

$$\theta_{k+1} = \theta_k + \omega_k \Delta t.$$

La función de coste usada por el script penaliza error de distancia, error al punto p^* , rumbo, esfuerzo, potencia de rueda y variación de comando:

$$\begin{aligned} J = \sum_{k=0}^{N-1} & (q_d e_d(k))^2 + q_t \|p_k^* - p_k\|^2 + q_\psi e_\psi(k)^2 \\ & + r_v v_k^2 + r_\omega \omega_k^2 + r_{whl} (\omega_{L,k}^2 + \omega_{R,k}^2) \\ & + s_v \Delta v_k^2 + s_\omega \Delta \omega_k^2 + q_f \|p_N^* - p_N\|^2. \end{aligned}$$

Los pesos usados en el script fueron $q_d = 48,0$, $q_t = 7,0$, $q_\psi = 1,2$, $r_v = 1,4$, $r_\omega = 1,8$, $r_{whl} = 0,020$, $s_v = 65,0$, $s_\omega = 18,0$ y $q_f = 6,0$, con horizonte $N = 10$, $\Delta t = 0,22$ s y dos iteraciones de mejora por ciclo. Esta implementación reside en `pioneer_pid_follower_controller.lua` y aplica exclusivamente a la escena de seguimiento (Actividad2_1_Basic_Follower.ttt). La escena warehouse (Sim_T2_Phase1_Basic.ttt) utiliza el controlador de tarea `phase1_r1_task_controller.lua`, donde el modo NMPC es una versión simplificada con ganancia adaptativa; por ello, la comparación de controladores P/PI/PID/LQR/NMPC presentada en este capítulo corresponde únicamente al follower.

La búsqueda local es determinista y de vecindario: la secuencia se inicializa con los comandos del ciclo anterior desplazados un paso, y en cada pasada se prueban perturbaciones $\pm \Delta v$ y $\pm \Delta \omega$ sobre cada elemento del horizonte. En la primera pasada los pasos son $\Delta v = 0.065$ y $\Delta \omega = 0.16$; en la segunda bajan a 0.025 y 0.065 . Solo se acepta una perturbación si reduce J . Por eso el coste es predictivo y el modelo es no lineal, pero el solucionador es una búsqueda local pequeña, no un SQP ni un método interior-point.

Los límites de actuación se aplican durante la búsqueda:

$$v_k \in [-v_{rev,max}, v_{max}], \quad \omega_k \in [-\omega_{max}, \omega_{max}].$$

Esta versión conserva el modelo no lineal y el coste predictivo, pero limita la optimización a una búsqueda local pequeña sobre comandos. El alcance queda limitado al método de disparo directo implementado en Lua.

4.6 Resultados de seguimiento

Modo	Error mov. [m]	Error final [m]	Energía [J]	RMS Δu	Flips
P	0,3542	0,0982	49,950	0,0475	11
PI	0,2011	0,1018	64,419	0,1318	16
PID	0,1915	0,0317	102,381	0,1759	26
LQR	0,0231	0,0865	92,287	0,0873	12
NMPC	0,0198	0,0887	77,174	0,0576	4

$n=1$ por modo. Energía: estimación cinemática aproximada (véase §4.2). Indicadores cualitativos.

Cuadro 4.1: Métricas del seguidor de Bill por modo de control.

Actividad 2.1 - Pioneer siguiendo a Bill en círculo: P, PI, PID, LQR y NMPC

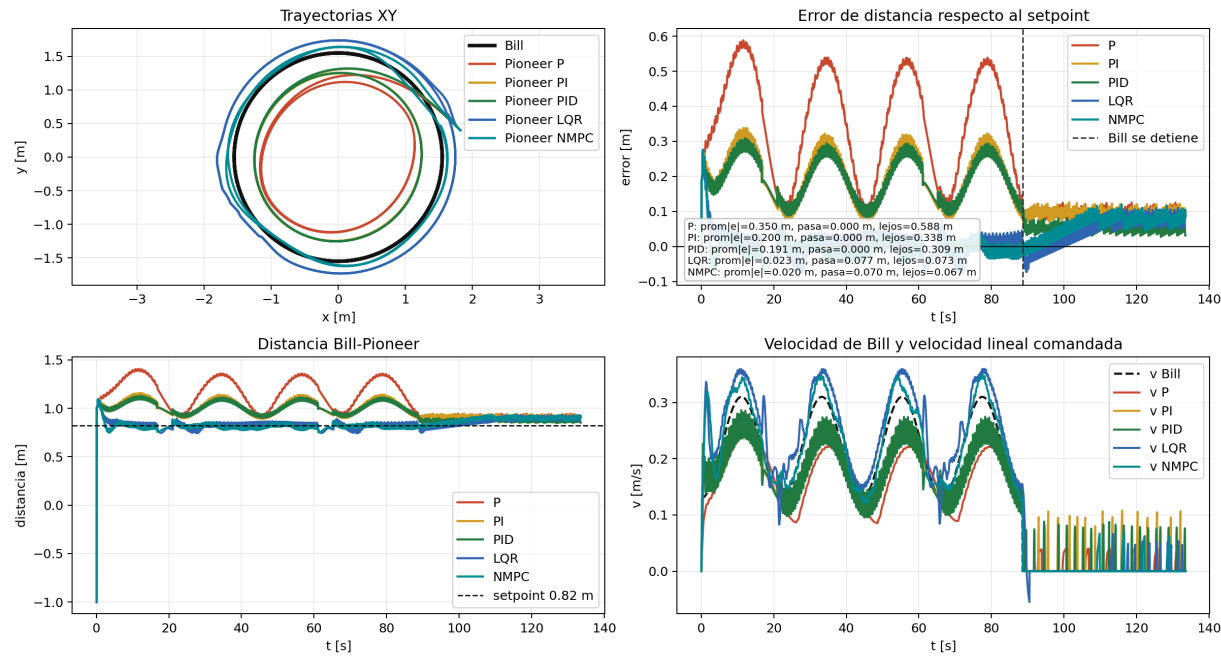


Figura 4.4: Comparación temporal de P, PI, PID, LQR y NMPC en la escena de seguimiento de Bill.

Actividad 2.1 - Datos de ruedas y potencia estimada

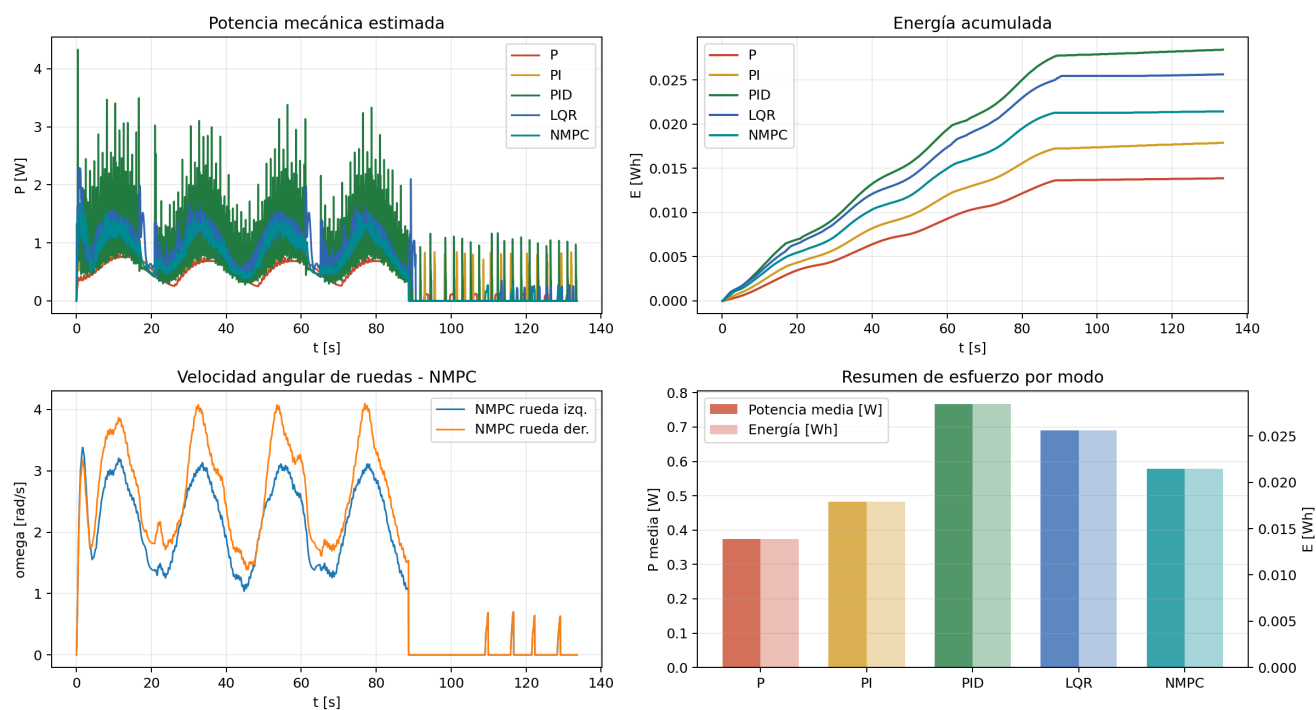


Figura 4.5: Coste físico aproximado del seguidor: potencia instantánea y energía acumulada en las ruedas.

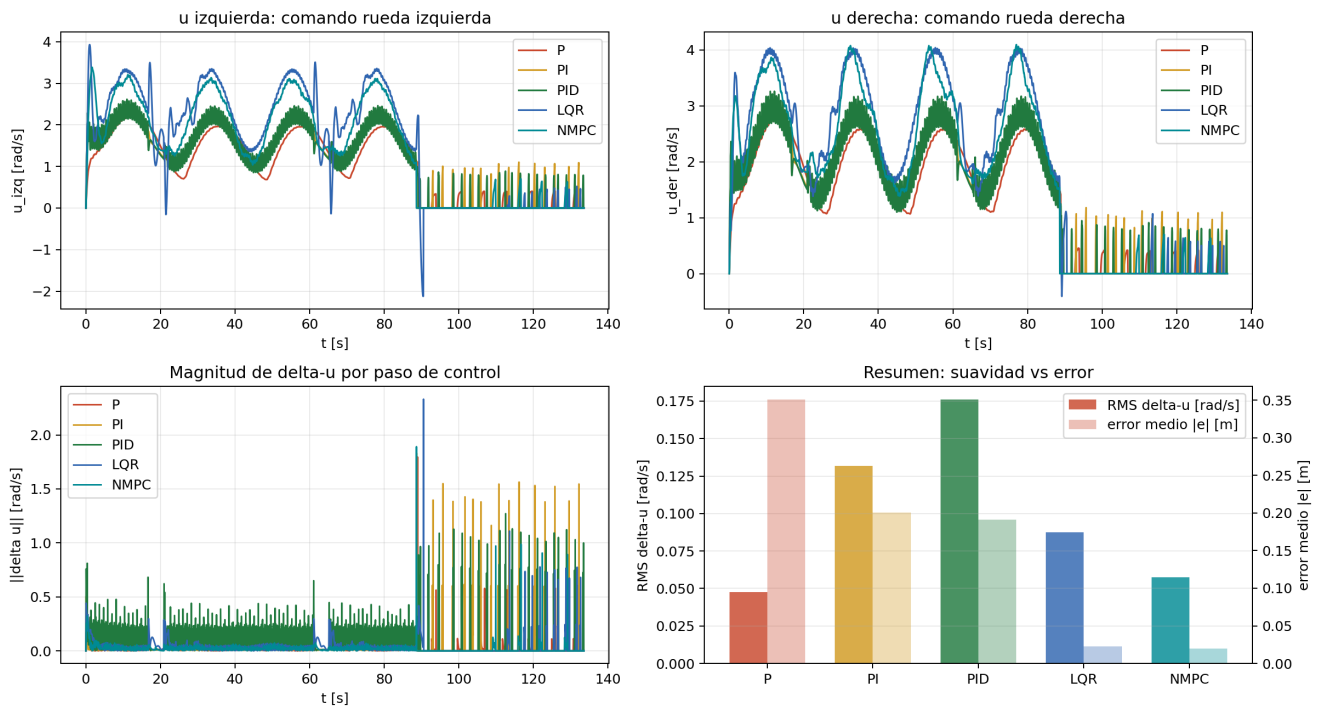
Actividad 2.1 - Suavidad de comandos de rueda u y Δu 

Figura 4.6: Suavidad de comandos del seguidor: variación de las órdenes de rueda y cambios de sentido.

La Fig. 4.4 muestra la respuesta temporal de los cinco modos; la Fig. 4.5 compara potencia y energía de rueda; la Fig. 4.6 compara la suavidad de comandos. P gasta menos energía y deja más error mientras Bill se mueve. PID corrige bien al final pero paga con más variación de comando y más energía. LQR y NMPC bajan el error medio durante el seguimiento; NMPC queda más suave en inversiones de sentido y RMS de Δu porque incluye en el coste la variación de comando y el esfuerzo de rueda.

5

Campos potenciales, evitación de obstáculos y planificación local

La navegación combina el campo atractivo hacia Bill, usado como mannequin, con una capa repulsiva de obstáculos y una capa local de mapa [8]. Los sensores de proximidad reducen la velocidad y corrigen el giro cuando se detecta un obstáculo al frente o en los laterales. El mapa SLAM entra cuando la ruta directa se bloquea y se usa un punto intermedio antes de volver al objetivo.

La palabra “campo” no significa que CoppeliaSim aplique una fuerza física al Pioneer. Es una forma de construir un vector de decisión: el objetivo tira del robot hacia Bill y cada obstáculo cercano empuja el rumbo hacia el lado libre. El resultado se traduce después a v y ω , que sí son las señales que acaban moviendo las ruedas.

5.1 Campo atractivo hacia Bill

La navegación define una fuerza de atracción hacia el mannequin. En la escena, Bill cumple ese papel. Si $p_R = [x_R, y_R]^T$ es la posición estimada del robot y $p_B = [x_B, y_B]^T$ la posición de Bill, se usa el potencial atractivo:

$$U_{att}(p_R) = \frac{1}{2}k_{att}\|p_B - p_R\|^2, \quad F_{att} = -\nabla U_{att} = k_{att}(p_B - p_R).$$

El controlador no aplica la fuerza como dinámica física; la traduce a consignas de avance y giro para el robot diferencial:

$$\theta_{att} = \text{atan2}(y_B - y_R, x_B - x_R), \quad e_\theta = \text{wrap}(\theta_{att} - \theta_R),$$

$$v_{att} = \text{sat}_{[0, v_{max}]}(k_v(\|p_B - p_R\| - d_f)), \quad \omega_{att} = k_\theta e_\theta.$$

La expresión anterior define la atracción hacia Bill. La repulsión de obstáculos modifica ω_{att} y limita v_{att} , pero no sustituye el objetivo principal de seguir a Bill.

5.2 Campo reactivo de obstáculos

Para cada sensor con detección positiva se calcula una influencia normalizada. La idea viene de campos potenciales artificiales y comportamiento Braitenberg [9, 10], aplicada a los sensores de proximidad del Pioneer:

El script base usa una tabla Braitenberg con pesos fijos por sensor. Se mantiene la relación sensor–motor y el peso depende de la distancia. Una lectura lejana apenas toca las ruedas; una lectura cercana fuerza más giro y reduce v .

$$\alpha_i = \begin{cases} \left(\frac{d_0 - d_i}{d_0} \right)^2, & 0 < d_i < d_0 \\ 0, & d_i \geq d_0 \end{cases}$$

donde d_i es la distancia medida y d_0 el rango de influencia. La contribución angular se aproxima con el lado del sensor:

$$\omega_{obs} = - \sum_i \text{sgn}(y_i) k_{rep} \alpha_i w_i$$

El factor w_i aumenta el peso de sensores frontales y reduce el de sensores traseros. El signo de y_i indica la dirección lateral del sensor en el marco del robot. Si un obstáculo está dentro de una distancia crítica frontal, el robot entra en modo AVOIDING, reduce v y gira en el sentido de menor riesgo.

La convención de signo usada en el script es la habitual del modelo unicycle: $\omega > 0$ incrementa el yaw en sentido antihorario y aumenta la velocidad de la rueda derecha respecto de la izquierda. Con $y_i > 0$ para sensores del lado izquierdo, el signo negativo de ω_{obs} hace que un obstáculo a la izquierda produzca giro horario, es decir, alejamiento hacia la derecha. Para sensores del lado derecho ocurre lo contrario.

5.3 Visualización de sensores

Los 16 sensores ultrasónicos del Pioneer se dibujan como rayos. Sin impacto, el rayo llega al rango nominal; con obstáculo, termina en el punto detectado. Esta visualización relaciona detección, giro reactivo y separación frente a pallet, pilar o caja.

5.4 Punto intermedio de planificación SLAM

En esta fase se usa una regla local de punto intermedio. Si un landmark o una celda ocupada cae demasiado cerca del segmento directo entre la pose estimada y el objetivo, se generan candidatos laterales a distancia d_{off} . Se elige el candidato con menor coste:

$$J(q) = \|q - \hat{x}_{R1}\| + \|x_g - q\| + \lambda C_{obs}(q)$$

donde C_{obs} penaliza candidatos cercanos a obstáculos mapeados. Si se activa este desvío, la señal `phase1PlannerMode` toma el valor `SLAM_WAYPOINT`; al alcanzar ese punto, el sistema vuelve a ruta directa.

6

Estimación y SLAM

El estimador sigue el ciclo clásico de predicción y corrección. Primero propaga la pose con odometría diferencial; después corrige esa pose con mediciones ruidosas del simulador y lleva las lecturas de sensor a coordenadas del warehouse. Con esas detecciones se actualiza un mapa compacto de landmarks o una grid de ocupación, según el algoritmo seleccionado [11].

La pose que entrega CoppeliaSim tiene seis componentes: $[x, y, z, \alpha, \beta, \gamma]$. El estimador usa solo la proyección plana $[x, y, \theta]$, con $\theta = \gamma$. La altura, roll y pitch se descartan porque el Pioneer permanece sobre el piso de la celda y la tarea se formula en 2D.

6.1 Sensores: LiDAR, ultrasonidos y rayos de proximidad

Un LiDAR 2D real emite muchos pulsos láser en ángulos conocidos. Para cada ángulo recibe una distancia, normalmente por tiempo de vuelo, y forma un escaneo polar:

$$\mathcal{S}_k = \{(r_i, \beta_i)\}_{i=1}^N.$$

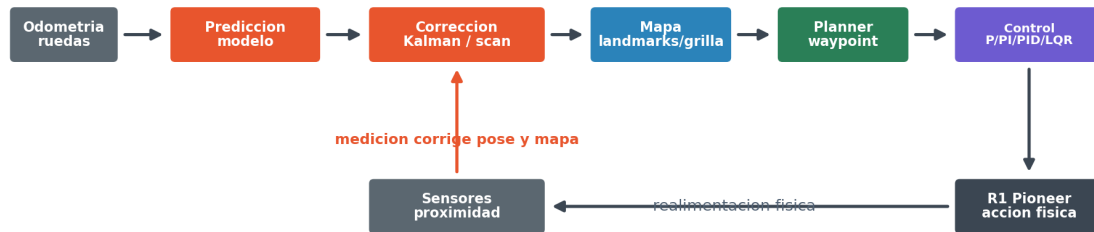
Cada par dice: “en el ángulo β_i , el primer obstáculo está a distancia r_i ”. Con la pose del robot, ese punto se transforma al marco global:

$$p_i^W = \begin{bmatrix} \hat{x} \\ \hat{y} \end{bmatrix} + R(\hat{\theta}) \begin{bmatrix} r_i \cos \beta_i \\ r_i \sin \beta_i \end{bmatrix}.$$

El Pioneer usado aquí no lleva un LiDAR físico 2D. La escena usa los 16 sensores ultrasónicos/proximidad del modelo P3DX, dibujados como rayos para que se vea qué detecta cada sensor. Conceptualmente se tratan igual que un escaneo de baja resolución: cada sensor tiene un ángulo fijo β_i , devuelve una distancia r_i si hay impacto y se transforma al mapa con la pose estimada. La diferencia está en la densidad y la precisión: un LiDAR aporta muchos más rayos y contornos más continuos; el arreglo del Pioneer da pocas direcciones y por eso los mapas de esta actividad son aproximaciones locales.

Esta aclaración importa para interpretar las figuras de SLAM. Cuando el trabajo habla de *scan*, rayos o reconstrucción, se refiere a los rayos de proximidad simulados, no a un LiDAR físico añadido al robot. Las variantes GMapping, Hector y Cartographer se implementan sobre esos datos para comparar representaciones, no para declarar equivalencia con sus paquetes ROS completos. La Fig. 6.1 muestra el ciclo completo desde sensores hasta control.

Ciclo estimacion - mapa - planificacion - control



Flujo principal: odometria -> estimacion -> mapa -> planner -> control -> accion. La vuelta cerrada regresa por sensores.

Figura 6.1: Secuencia de estimación, mapa, planificación local y control de R1.

6.2 Datos usados para comparar SLAM

Los cuatro métodos reciben las mismas entradas: odometría diferencial, pose ruidosa de referencia para cerrar el estimador, rayos de proximidad y el mismo objetivo de navegación. La comparación no mide paquetes ROS completos, sino cuatro representaciones implementadas en Lua para la misma celda: rasgos compactos, grid de ocupación, ajuste local por scan matching y submapas.

Nota sobre el RMSE reportado. La medición de pose que cierra el estimador en cada variante SLAM no viene de un sensor físico ni de un generador gaussiano aleatorio: se toma la pose real plana del simulador (`sim.getObjectPosition`) y se le añade una perturbación sinusoidal acotada. Por consiguiente, el RMSE de posición mide la distancia entre la estimación filtrada y esa misma pose real perturbada, quedando acotado por la amplitud del ruido inyectado. Los valores reportados reflejan la estabilidad del estimador bajo esas condiciones reproducibles, no la calidad de localización frente a un sensor autónomo real.

La medición de pose difiere intencionalmente entre métodos:

- **KALMAN_LANDMARK:** amplitud 0,030 m en x, y y 0,018 rad en yaw. El filtro de Kalman propaga covarianza y puede manejar esta corrección de forma robusta.
- **Métodos grid (GMAPPING, HECTOR, CARTOGRAPHER):** amplitud 0,042 m en x, y y 0,026 rad en yaw. Estas variantes usan corrección de ganancia fija ($k_{xy} = 0,16$, $k_{\theta} = 0,12$); con ruido menor la corrección producía oscilaciones, por lo que se aumentó la amplitud para amortiguar el estimador.

Esta asimetría significa que **Kalman recibe un 40 % menos de ruido de pose** que los métodos grid, lo que contribuye a su mejor RMSE. La comparación mide las representaciones de mapa bajo sus

condiciones de operación estables, no bajo condiciones idénticas de ruido. Para una comparación con ruido equitativo sería necesario ajustar el estimador de corrección de los métodos grid.

La covarianza de medición que usa la corrección Kalman queda parametrizada con $\sigma_{xy} = 0,055$ m y $\sigma_\theta = 0,045$ rad. Para detecciones de obstáculo se usaron $\sigma_r = 0,075$ m y $\sigma_\beta = 0,10$ rad. Esto mantiene repetible la comparación; no pretende modelar el ruido exacto de un sensor real.

La diferencia está en qué guarda cada algoritmo y cómo eso afecta al planificador. Kalman-landmark guarda pocos puntos fusionados; GMapping guarda probabilidades por celda; Hector corrige la pose buscando el scan que mejor encaja con la grid; Cartographer divide el mundo en submapas y registra cierres de ciclo. Por eso la comparación no se reduce a “número de rasgos”: un landmark y una celda ocupada no tienen el mismo significado físico.

6.3 Por qué hace falta Kalman

La odometría diferencial predice bien durante pocos instantes, pero acumula error: un pequeño desajuste de rueda, saturación o giro se convierte en deriva de x, y, θ . Las mediciones de sensor tampoco son perfectas; un impacto puede aparecer desplazado por ruido, discretización o por estar mirando un borde. El filtro de Kalman se usa para combinar esas dos fuentes en vez de creer ciegamente en una sola.

La idea práctica es simple. Primero se predice dónde debería estar el robot con el modelo de movimiento y los comandos de rueda. Después llega una medición ruidosa y se calcula cuánto corregir. Si la predicción tiene mucha incertidumbre P , la ganancia K da más peso a la medición. Si la medición es poco fiable R , K corrige menos. En la escena esto mantiene una pose suave para el controlador y evita que un rayo aislado mueva el mapa de forma brusca.

En esta actividad Kalman no se usa porque sea el único método posible, sino porque encaja con una celda pequeña y con pocos obstáculos: mantiene una pose estable, fusiona detecciones cercanas como landmarks y entrega al planificador una lista compacta de puntos peligrosos. Su límite es claro: no dibuja paredes ni superficies completas como una grid de ocupación.

La arquitectura de corrección combina dos fuentes de información. Primero, `fusePoseMeasurement` aplica cada ciclo una corrección de pose a partir de la posición del simulador perturbada con ruido sinusoidal acotado (0,030 m en x, y y 0,018 rad en yaw); esta señal actúa como referencia GPS sintética reproducible. Segundo, cada vez que un sensor detecta un obstáculo, `updateSlamLandmark` corrige la pose del robot y las posiciones de los landmarks de forma conjunta mediante el jacobiano de observación H y la actualización de covarianza EKF, con peso `slamPoseCorrectionWeight = 1,0`. Esta segunda corrección introduce la información de sensor en la pose y cierra el ciclo de estimación sin depender exclusivamente de la referencia externa. La combinación de ambas fuentes corresponde a un esquema EKF asistido por GPS sintético, común en entornos de simulación donde la pose del simulador sirve como punto de referencia de validación [11].

6.4 Predicción de pose

Con velocidades odométricas v_k, ω_k , el modelo diferencial discreto es:

$$\begin{aligned}\hat{x}_k^- &= \hat{x}_{k-1}^+ + v_k \cos(\hat{\theta}_{k-1}^+) \Delta t \\ \hat{y}_k^- &= \hat{y}_{k-1}^+ + v_k \sin(\hat{\theta}_{k-1}^+) \Delta t \\ \hat{\theta}_k^- &= \text{wrap}(\hat{\theta}_{k-1}^+ + \omega_k \Delta t)\end{aligned}$$

La covarianza se propaga con el Jacobiano G_k del modelo de movimiento:

$$G_k = \begin{bmatrix} 1 & 0 & -v_k \Delta t \sin \hat{\theta}_{k-1}^+ \\ 0 & 1 & v_k \Delta t \cos \hat{\theta}_{k-1}^+ \\ 0 & 0 & 1 \end{bmatrix}, \quad P_k^- = G_k P_{k-1}^+ G_k^\top + Q_k$$

6.5 Corrección Kalman-landmark

La formulación de referencia sigue el filtro de Kalman extendido (EKF) para sistemas de navegación con landmarks [11]:

$$K_k = P_k^- H_k^\top (H_k P_k^- H_k^\top + R_k)^{-1}$$

$$\hat{x}_k^+ = \hat{x}_k^- + K_k(z_k - H_k \hat{x}_k^-), \quad P_k^+ = (I - K_k H_k) P_k^-$$

En la escena, la pose se corrige con una medición ruidosa generada desde CoppeliaSim, y las detecciones de obstáculo se asocian por cercanía. Si una lectura cae dentro del radio de asociación de un landmark, esa landmark se actualiza con una ganancia escalar; las lecturas no asociadas crean landmarks hasta el máximo configurado. KALMAN_LANDMARK se denomina así porque no estima una covarianza conjunta entre pose y mapa, como sí haría un EKF-SLAM completo (Fig. 6.2).

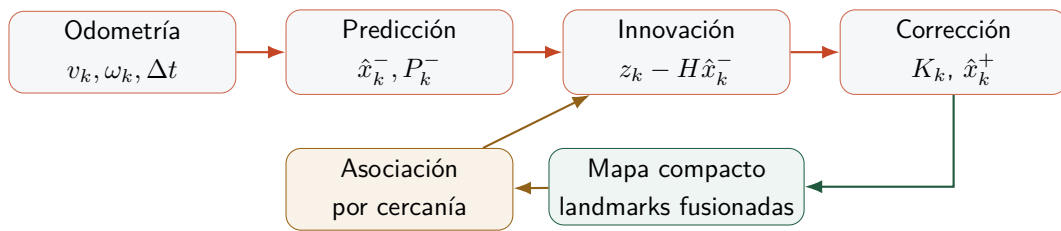


Figura 6.2: Particularidad de Kalman-landmark: corrige la pose y fusiona pocas detecciones como rasgos; no mantiene una covarianza conjunta pose–mapa como un EKF-SLAM completo.

En un entorno pequeño, esta opción mantiene estabilidad: al fusionar lecturas cercanas, el mapa no crece sin control y el planificador recibe obstáculos puntuales fáciles de filtrar. La desventaja es visual: no reconstruye superficies completas, de modo que una pared larga aparece como pocos rasgos y no como una frontera continua.

6.6 GMapping en grid

GMapping se basa en filtros de partículas Rao-Blackwellizados y mapas de grid [13]. En esta simulación implementamos una grid de ocupación con log-odds, donde $L(c) = \log(p(c)/(1 - p(c)))$. Para cada rayo, las celdas atravesadas se marcan como libres y la celda de impacto como ocupada:

$$L_t(c) = \text{clip} \left(L_{t-1}(c) + \begin{cases} l_{occ}, & c = c_{hit} \\ l_{free}, & c \in ray \end{cases} \right)$$

Los incrementos son $l_{occ} = 0,85$ y $l_{free} = -0,18$; se acotan con $\text{clip}(\cdot, -3,60, 3,60)$ para mantener las probabilidades en el rango $[0,027, 0,973]$ y evitar saturación por lecturas aisladas. Una celda se considera ocupada si $L(c) \geq 1,05$ (parámetro `gridOccupiedThreshold`).

La grid resultante se usa tanto para telemetría como para el planificador local. En Lua, el recorrido de celdas se obtiene muestreando el segmento entre el sensor y el punto detectado con paso menor que la resolución de la grid. Es una variante simple de ray-casting, suficiente para marcar las celdas libres antes de la celda de impacto (Fig. 6.3).

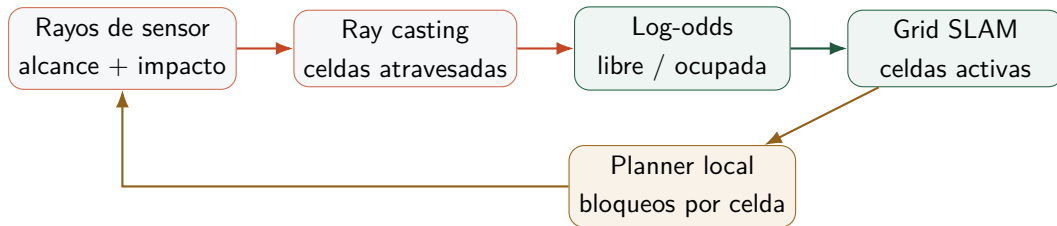


Figura 6.3: Particularidad de GMapping implementado: la información principal es una grid de ocupación. Cada rayo libera celdas antes del impacto y refuerza la celda ocupada.

El GMapping original usa un filtro de partículas Rao-Blackwellizado para mantener hipótesis de trayectoria. En esta actividad se conservó la idea que interesa para el planificador, la grid de ocupación, y se redujo el resto para que el coste computacional fuera razonable dentro del script de CoppeliaSim. Esa regla explica por qué GMapping genera muchos más elementos de mapa que Kalman-landmark, pero no necesariamente menor error de pose.

6.7 Hector con ajuste local

Hector SLAM usa *scan matching* contra una grid y depende menos de la odometría [14]. En esta escena implementamos una búsqueda local discreta alrededor de la pose estimada. Para cada candidato (x, y, θ) se proyectan los puntos del scan y se evalúa el puntaje contra la grid:

$$s(x, y, \theta) = \frac{1}{N} \sum_{i=1}^N L(x + r_i \cos(\theta + \beta_i), y + r_i \sin(\theta + \beta_i))$$

Si el mejor puntaje supera el umbral ($s > 0,45$), la pose se corrige parcialmente (ganancia 0,18) hacia ese candidato. `HECTOR_GRID_MATCHING` usa una búsqueda discreta de $5 \times 5 \times 5 = 125$ candidatos sobre $(\Delta x, \Delta y, \Delta \theta)$ para que el cálculo quepa dentro del script Lua. Hector en ROS trabaja con Gauss-Newton

multi-resolución; se usa una versión reducida y revisable para esta escena sin minimización de gradiente (Fig. 6.4).

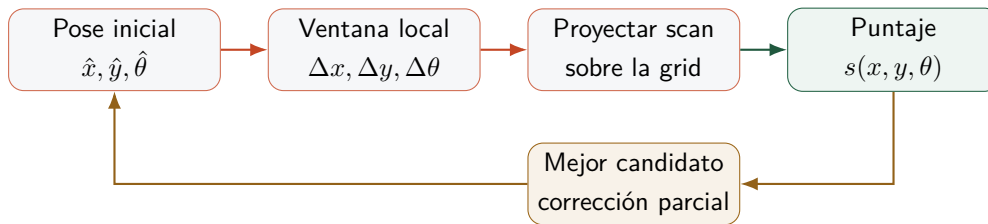


Figura 6.4: Particularidad de Hector: busca la pose local cuyo scan encaja mejor con la grid. La odometría inicializa la búsqueda, pero la corrección viene del ajuste scan–mapa.

Esto hace que Hector sea útil cuando los rayos tienen geometría suficiente para alinearse con el mapa. Si el entorno observado es pobre, simétrico o con pocos impactos, varios candidatos reciben puntajes parecidos y la corrección tiende a oscilar. Por esa razón el script aplica solo una fracción de la corrección y mantiene umbrales para no mover la pose ante coincidencias débiles.

6.8 Cartographer con submapas

Google Cartographer organiza el mapeo en submapas y cierres de ciclo para reducir deriva acumulada [15]. La implementación local mantiene submapas temporales, crea submapas adicionales por distancia o periodo y aplica un cierre de ciclo sencillo cuando la pose vuelve cerca de un submapa antiguo con puntaje por encima del umbral. En la comparación actual produjo 13 submapas y 4 cierres de ciclo.

En concreto, un submapa se considera antiguo si no es el activo ni el anterior y si han pasado más de 22s desde su creación. El cierre se acepta solo si la distancia al submapa es menor que 0,45m, hay al menos tres submapas de separación, el puntaje de coincidencia de scan supera 0.35 y no se ha registrado ya ese enlace. La corrección aplicada es parcial (10% en x, y , 6% en yaw), precisamente para evitar que un falso positivo rompa la trayectoria.

Nota de implementación. Hector y Cartographer comparten el mismo mecanismo de búsqueda de pose en Lua (`applyGridScanMatching`): búsqueda discreta de $5 \times 5 \times 5 = 125$ candidatos con diferentes radios de búsqueda (0,075m y 0,065rad para Hector; 0,055m y 0,045rad para Cartographer). La diferencia algorítmica entre ambos, en esta implementación, es únicamente el radio de búsqueda y la gestión adicional de submapas en Cartographer; no se implementa la optimización Gauss-Newton ni el ajuste de grafo de poses del Cartographer original [15].

El flujo de submapas y cierres de ciclo se ilustra en la Fig. 6.5.

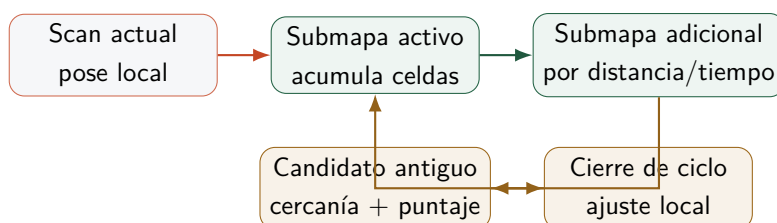


Figura 6.5: Particularidad de Cartographer-submap: el mapa no se guarda como una sola grid monolítica, sino como submapas locales con posibles cierres cuando R1 vuelve a zonas conocidas.

El mapa conserva estructura temporal: registra qué zona se construyó en cada tramo y detecta retornos a regiones ya visitadas. El coste es una lógica más delicada para navegación local. Si una celda antigua queda cerca de la ruta, el planificador la sobreinterpreta como bloqueo; por eso en Phase 2 se añadieron filtros de persistencia, distancia al robot y recuperación directa.

6.9 Qué método usar y por qué

Los cuatro métodos responden a preguntas distintas (Tabla 6.1). Kalman-landmark pregunta “¿dónde está el robot y qué puntos cercanos debo evitar?”. GMapping pregunta “¿qué celdas del plano están libres u ocupadas?”. Hector pregunta “¿qué pequeña corrección de pose hace que el escaneo actual encaje mejor con el mapa?”. Cartographer pregunta “¿cómo divido el recorrido en submapas y reconozco zonas visitadas de nuevo?”.

Método	Conviene cuando	No conviene cuando	Lectura en esta actividad
Kalman-landmark	Se necesita pose estable y pocos obstáculos puntuales.	Se requiere una pared o contorno continuo del entorno.	Mejor RMSE y menor duración; opción base para la celda compacta.
GMapping/log-odds	El planificador necesita saber qué celdas están ocupadas.	Se espera estimación completa de trayectoria por partículas.	Útil como grid de ocupación ligera; no implementa el GMapping completo.
Hector/scan-to-map	Hay suficiente geometría para alinear el scan contra el mapa.	Los rayos son pocos, simétricos o con impactos pobres.	Da buena precisión de rasgos en Phase 2, pero puede oscilar si el scan no discrimina candidatos.
Cartographer	El robot recorre zonas amplias y vuelve a regiones ya vistas.	La escena es pequeña y el coste de gestionar submapas supera el beneficio.	Logra mayor cobertura y cierres, pero tarda más y exige filtros para no bloquear la ruta.

Cuadro 6.1: Criterio de selección entre las variantes SLAM implementadas.

Con los datos de la actividad, la elección operativa es Kalman-landmark para controlar el Pioneer en esta celda. Las grids y submapas se mantienen para comparar cómo cambia el mapa cuando se prioriza ocupación, ajuste local o estructura temporal.

7

Resultados experimentales

Los resultados proceden de ejecuciones de CoppeliaSim lanzadas desde Python. Cada ejecución exporta señales a CSV y JSON, y de ahí salen las tablas y figuras. Las capturas renderizadas desde CoppeliaSim complementan las gráficas y vinculan cada métrica con la escena simulada.

7.1 Validación funcional de la escena

La validación principal de `Sim_T2_Phase1_Basic.ttt` terminó con `passed=true`. Los indicadores más relevantes fueron:

Indicador	Resultado
Estado final de tarea	CHARGING
Modo final de movimiento	ARRIVED_TARGET
Tareas completadas	4/4
Distancia caminada por Bill	18,611 m ^a
Batería final R1	40,5 % ^a
Sensores activos	16
Rayos de sensor visualizados	16
Landmarks/celdas SLAM finales	11 (Kalman-landmark básico) ^a
Actualizaciones SLAM	967 ^a
Error final de pose	0,034 m ^a
Error máximo de pose	0,049 m ^a
Distancia mínima positiva a obstáculo	0,056 m
Riesgo máximo de obstáculo	0,829

^a Valor extraído de `Sim_T2_Phase1_Basic_validation.json`. El comparativo SLAM extendido usa configuración distinta (más actuali

Junto con las métricas, se comprobó la ejecución en la escena: Bill se mueve entre WS1 y WS2, toma y entrega piezas, trabaja en la mesa, aparecen estanterías, mesas y sofá, T1 y T2 tienen geometría reconocible, la cola de tareas queda registrada, el robot vuelve a carga y el planificador activa `SLAM_WAYPOINT` cuando detecta un bloqueo local. La FSM de tarea recorre 19 cadenas de estado distintas durante la misión completa (listadas en `task_states_seen` del JSON de validación); el diagrama de la presentación muestra los 9 estados de tarea principales, omitiendo los estados intermedios de navegación (`PICK_RETURN_T1_WS1`, `WAIT_B1_WS2_REQUEST_T2`, `TO_WS1_DELIVER_T1`, etc.).

7.2 Desempeño del estimador Kalman-landmark

En la exportación específica del estimador, la escena usa el modo PID del controlador de tarea (`phase1_r1_task_controller.lua`) como modo base y `KALMAN_LANDMARK` como algoritmo SLAM. El modo PID de este controlador de tarea usa términos proporcional y derivado sobre el error de distancia, sin término integral explícito; la comparación de controladores con integral completa (P/PI/PID/LQR/NMPC) se realizó con el controlador de seguimiento del follower (`pioneer_pid_follower_controller.lua`) en la escena separada de seguimiento. Los resultados de la exportación del estimador en el warehouse fueron (nótese que los errores de pose difieren respecto a la Tabla 7.2 porque ambas exportaciones usan configuraciones distintas; véase la nota al final de la sección):

- RMSE de posición: 0,03142 m.
- Error medio de posición: 0,02981 m.
- Error P95: 0,04760 m.
- Error máximo: 0,05626 m.
- RMSE de orientación: 0,01987 rad.
- Covarianza media estimada: 0,00081.
- Landmarks finales: 20.
- Actualizaciones Kalman/SLAM: 4295.

El número de elementos del mapa depende de la representación. En Kalman-landmark son rasgos puntuales fusionados; en las variantes de grid son celdas ocupadas o grupos de celdas. La validación funcional cerró con 24 rasgos y la exportación específica Kalman-landmark con 20, mientras que las grids acabaron con mapas más densos (Fig. 7.1).

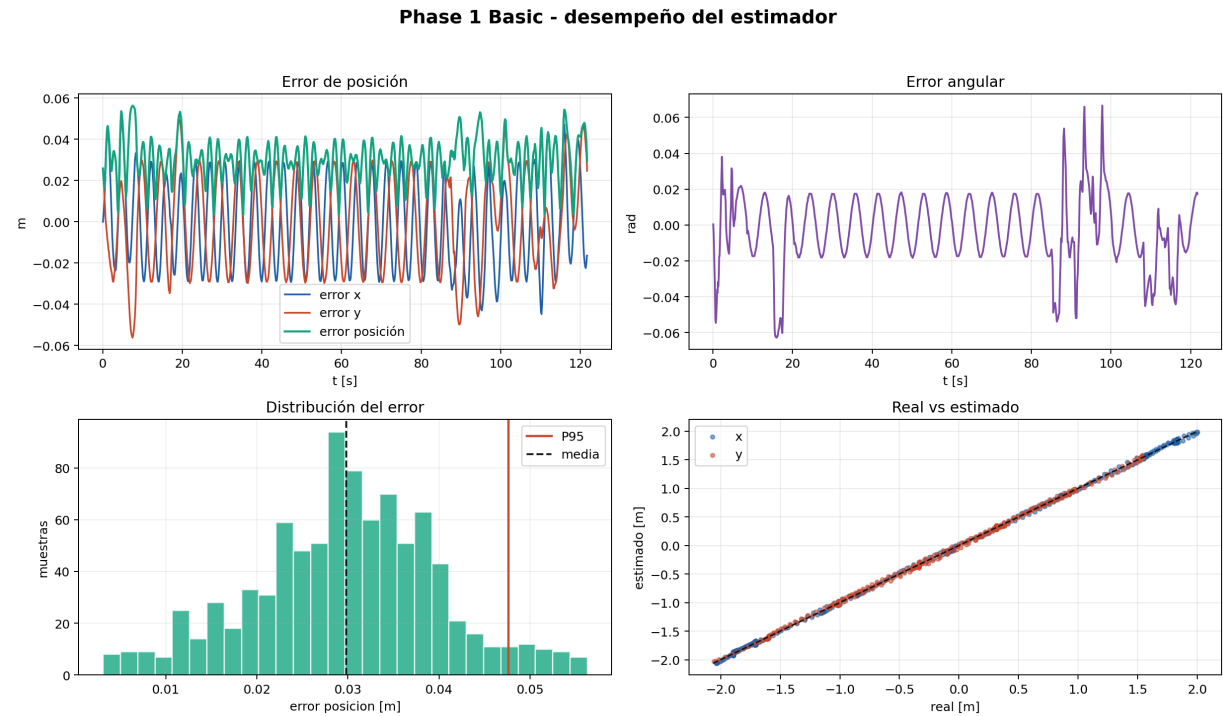


Figura 7.1: Resultados del estimador Kalman-landmark.

7.3 Comparación de algoritmos SLAM

La comparación usa el mismo escenario, sensores, tareas y controlador para los cuatro modos de SLAM. Los valores de batería del comparativo SLAM ($\approx 71\text{--}72\%$ restante) difieren de los de la validación básica ($40,5\%$) porque ambas ejecuciones usan configuraciones distintas: la validación básica corre el validador funcional con el SLAM compacto predeterminado (967 actualizaciones), mientras que el comparativo SLAM extiende la exportación de métricas durante toda la misión (4275–4514 actualizaciones) con mayor período de registro y forzado de ciclos de actualización. Ambas completan las 4 tareas correctamente. La Tabla 7.2 recoge los valores principales.

Los tiempos y longitudes de ruta salen parecidos porque el plan de misión, los puntos de entrega y el controlador de movimiento son los mismos. Cambia la representación del mapa usada por los bloqueos y los puntos intermedios. Por eso la columna de rasgos exige cuidado: en Kalman son landmarks fusionadas; en las grids son celdas o grupos de celdas ocupadas.

Cuadro 7.2: Comparación SLAM en CoppeliaSim.

Métrica	GMapping	Hector	Cartographer	Kalman
Completado	Sí	Sí	Sí	Sí
Duración [s]	125,60	123,55	125,25	121,45
Ruta [m]	15,997	15,233	15,968	15,763
RMSE pos. [m]	0,04117	0,04445	0,04149	0,03142
Error P95 [m]	0,05718	0,07596	0,06026	0,04670
Error max. [m]	0,07354	0,10862	0,07362	0,05824
Rasgos finales	94	84	91	20

Métrica	GMapping	Hector	Cartographer	Kalman
Actualizaciones SLAM	4514	4470	4474	4275
Submapas	0	0	13	0
Cierres de ciclo	0	0	4	0
Batería usada [%]	24,62	23,63	24,50	24,02

$n=1$ ejecución por método. Los resultados son indicadores cualitativos; no constituyen comparación estadísticamente validada.

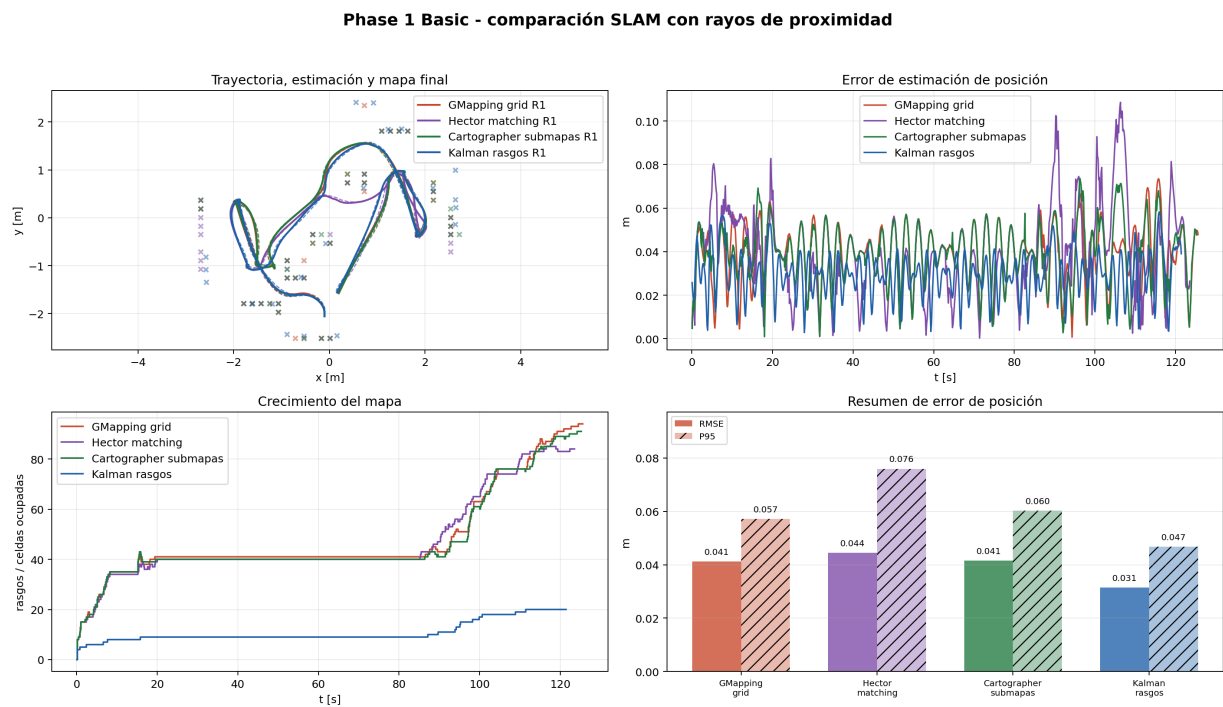


Figura 7.2: Comparación gráfica de los cuatro modos SLAM: trayectoria, precisión, crecimiento del mapa y resumen de error.

7.4 Interpretación de resultados

La Fig. 7.2 sintetiza los resultados gráficamente. Kalman-landmark obtuvo el menor error de pose (RMSE 0,031 m) y la menor duración de misión. Hector registró el menor consumo de batería (23,63 %) y la ruta más corta entre las variantes grid (15,233 m). GMapping genera un mapa más denso que Hector sin scan matching. Cartographer redujo la ruta frente a GMapping (15,968 m vs. 15,997 m) y registró estructura por submapas con 4 cierres de ciclo locales, aunque su RMSE quedó por encima de Kalman-landmark y su ruta fue superior a la de Hector. En una celda pequeña, las landmarks fusionadas de Kalman son suficientes para navegar con error mínimo.

Para la escena base usamos KALMAN_LANDMARK como estimador base. Los otros tres métodos quedan para comparar mapas calculados con los mismos rayos de proximidad.

8

Phase 2: SLAM con mapa desconocido

La escena `Sim_T2_Phase2_SLAM_Unknown.ttt` parte de la celda validada y arranca con parte del entorno oculto. R1 ve cajas, paredes, una tarima móvil y un bloque temporal a través de los sensores de proximidad dibujados como rayos. El controlador actualiza el mapa que va armando y lo usa para activar puntos intermedios locales.

Solo `P2_Dynamic_Pallet` y `P2_Temporary_Blocker` se mueven por script, con trayectorias periódicas deterministas. Las crates y los marcadores de zona desconocida quedan fijos.

La métrica de actividad de mapa en Phase 2 (`phase2MappingEvidencePct`) cuenta rasgos detectados, celdas ocupadas y actualizaciones SLAM acumuladas durante la misión. Una mayor actividad no implica necesariamente mejor geometría; por eso la cobertura contra la referencia geométrica se revisa aparte con exhaustividad de obstáculos y precisión de rasgos frente a esos obstáculos.

Nota sobre los tiempos de cobertura. Los datos del comparativo muestran que los métodos de grid alcanzan el 50 % de actividad de mapa en tan solo $\approx 1,5$ s tras el inicio de la simulación, mientras que Kalman-landmark tarda $\approx 19,8$ s en el mismo umbral. Esta diferencia no refleja velocidad de exploración por navegación: la actividad de mapa para métodos de grid se dispara desde el primer ciclo porque los rayos de proximidad ya barren la zona inicialmente desconocida desde la posición de partida del robot (cobertura por campo de visión, no por movimiento). Kalman tarda más en acumular la misma fracción porque sus actualizaciones son discretas (20 landmarks máx.). El valor de 94–95 % al final de la misión sí refleja cobertura acumulada con navegación completa de la celda.



(a) Inicio con zona desconocida.

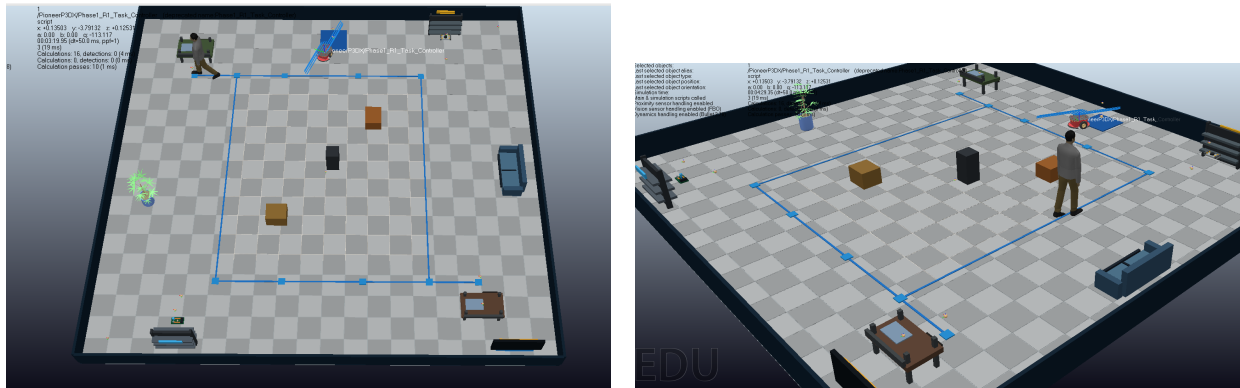


(b) Replanificación ante obstáculo dinámico.

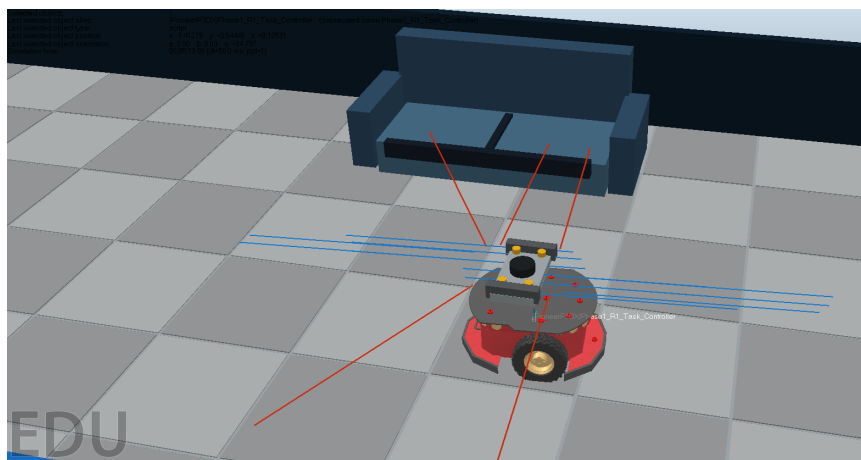
Figura 8.1: Capturas 3D de la escena Phase 2 ejecutada en CoppeliaSim.

Las capturas operativas se recogen en la Fig. 8.1; el conjunto extendido con lecturas de sensores y

plano general de la celda se muestra en la Fig. 8.2.



(a) Zona desconocida, Bill y primer bloque no modelado. (b) Plano general de la ruta y los elementos de la celda.



(c) Lecturas de proximidad frente al sofá.

Figura 8.2: Capturas de ejecución de Phase 2 con mapa desconocido y obstáculos en la celda.

8.1 Recuperación ante bloqueo local

En Phase 2, las variantes de grid podían quedar atrapadas en estados de recogida, con alternancia prolongada entre AVOIDING y SLAM_WAYPOINT. El planificador trataba como bloqueo cualquier celda ocupada cerca del segmento al objetivo, incluso cuando la celda estaba pegada al robot, a la estantería o al punto de recogida.

El planificador aplica cinco reglas para evitar ese ciclo:

- Las celdas de grid necesitan varias detecciones antes de influir en la planificación.
- Los obstáculos muy cercanos al robot o al objetivo no fuerzan un punto intermedio.
- Los candidatos de desvío reciben penalización si salen del área útil del warehouse.
- Tras alcanzar un punto intermedio, R1 mantiene una ventana breve de ruta directa para evitar replanificar contra el mismo obstáculo.
- Si una variante de grid se queda varios segundos sin reducir distancia al objetivo, se activa RECOVERY_DIRECT: la planificación ignora el mapa durante una ventana corta y conserva la evitación reactiva.

Con estas reglas los cuatro modos completan la misión en Phase 2. En la ejecución final, Cartographer activó una recuperación del planificador; GMapping, Hector y Kalman terminaron sin usarla.

8.2 Comparación de métodos

En Phase 2 los cuatro métodos se someten al mismo problema: arrancar con mapa incompleto, descubrir obstáculos no modelados y seguir navegando mientras una tarima se mueve. La comparación no se ordena por cantidad de mapa, sino por la representación que mantiene la misión sin bloqueo:

Kalman-landmark prioriza pose estable y pocos rasgos; **GMapping** prioriza ocupación por celda; **Hector** intenta corregir pose por coincidencia scan-grid; **Cartographer** conserva submapas y cierres de ciclo, pero introduce más condiciones para evaluar si una celda antigua bloquea o no la ruta.

Cuadro 8.1: Comparación Phase 2 con mapa desconocido y obstáculos dinámicos.

Métrica	GMapping	Hector	Cartographer	Kalman
Completado	Sí	Sí	Sí	Sí
Duración [s]	128,00	129,40	138,65	124,80
Ruta [m]	15,905	15,735	16,441	15,752
RMSE pos. [m]	0,04220	0,04587	0,03930	0,03185
Error P95 [m]	0,05638	0,07590	0,05773	0,03985
Actividad de mapa [%]	94,851	94,930	95,327	94,613
Rasgos finales	36	36	36	24
Exhaustividad aprox. final	1,000	1,000	1,000	1,000
Precisión aprox. final	0,250	0,333	0,278	0,250
Replanificaciones	19	24	25	17
Recuperaciones del planificador	0	0	1	0
Batería usada [%]	24,576	24,476	25,916	24,324
Submapas	0	0	13	0
Cierres de ciclo	0	0	4	0

La coincidencia de 13 submapas y 4 cierres entre Fase 1 y Fase 2 se debe a que se reutilizó la misma parametrización de Cartographer: longitud máxima de submapa, edad mínima de submapa y umbral de cierre idénticos en ambas escenas.

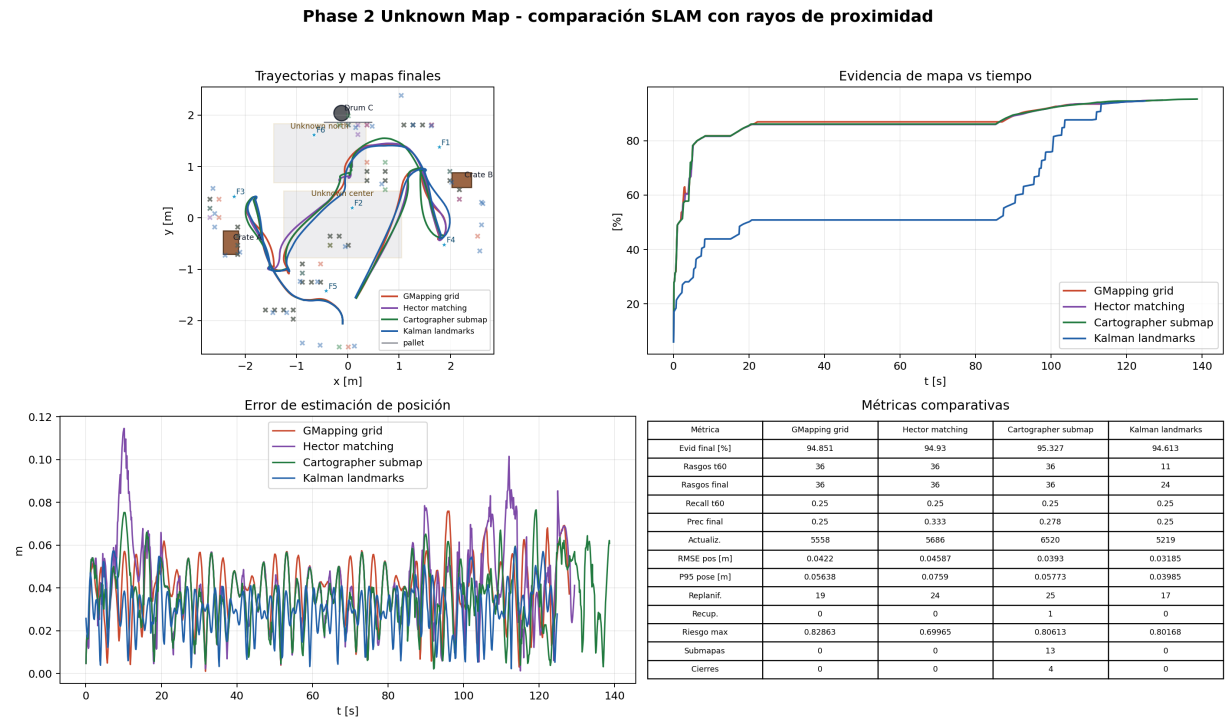


Figura 8.3: Comparación de métodos en Phase 2.

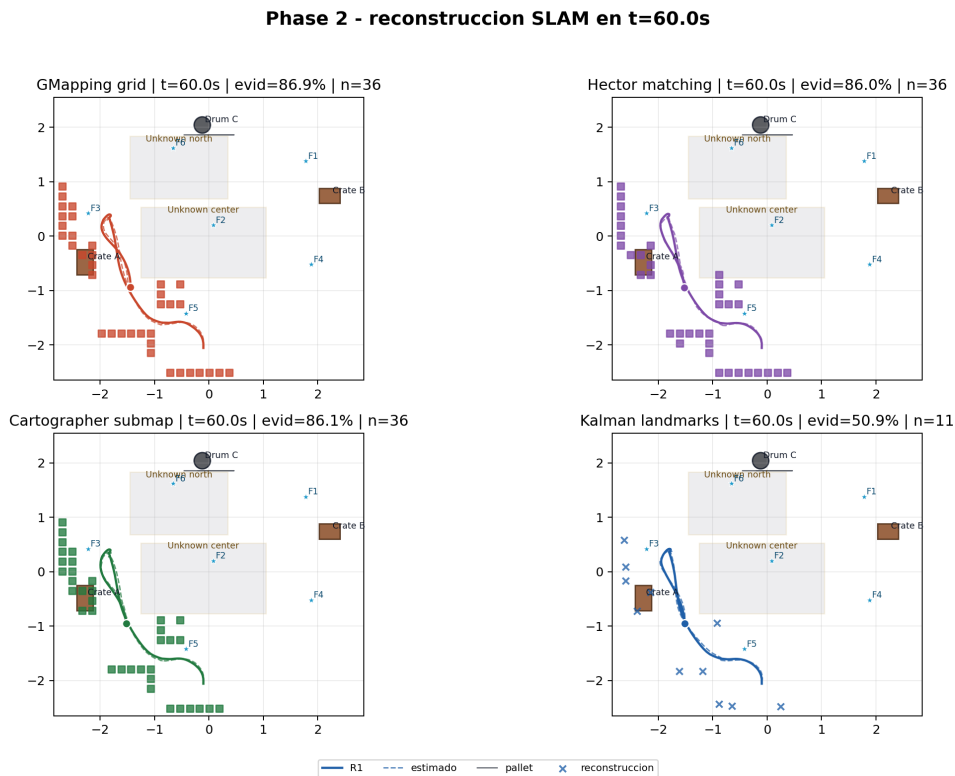


Figura 8.4: Reconstrucción del mapa al mismo instante común, $t = 60$ s.

8.3 Discusión técnica

La Tabla 8.1 recoge los valores numéricos completos; las Figs. 8.3 y 8.4 los resumen gráficamente junto con la reconstrucción del mapa. Kalman-landmark fue el modo operativo más eficiente: terminó antes, usó menos batería y mantuvo el menor RMSE de pose. Su mapa es compacto, suficiente para la navegación de esta celda, aunque menos denso que las representaciones de grid.

Los modos de grid generan más rasgos de mapa y aumentan rápido la actividad de mapeo. Esa métrica cuantifica actividad, no calidad geométrica por sí sola. Cartographer fue la mejor grid en RMSE y actividad final, aunque necesitó más ruta, más replanificaciones y una activación de RECOVERY_DIRECT. Hector obtuvo la mayor precisión aproximada final entre las grids y también registró el mayor error máximo de pose. GMapping completó la misión con menor duración que las otras grids.

Para ejecutar la misión con bajo error de pose, Kalman-landmark queda como base. Para representar ocupación, las grids generan mapas más densos, pero su utilidad debe evaluarse junto con error de pose, ruta, replanificaciones y recuperaciones.

9

Validación

La validación revisa el comportamiento del sistema y los archivos generados. La lista de verificación cubre escena, objetos, scripts integrados, tareas completadas, movimiento de Bill, acciones visuales, anticolisión, error del estimador y rayos de sensor. Las capturas 3D se obtuvieron desde CoppeliaSim con `capture_coppelasim_screenshots.lua`; son las imágenes de las Figuras 2.2 y 8.1.

9.1 Validación por lista de verificación

Bloque	Verificación	Estado
Escena	Existe <code>Sim_T2_Phase1_Basic.ttt</code> y contiene R1, B1, T1, T2, C1.	OK
Mannequin	B1 (Bill) se usa como persona móvil tipo <code>People/mannequin</code> .	OK
Warehouse	Dos mesas, dos estanterías, sofá, obstáculos y panel de tareas.	OK
Scripts	Controlador R1 y manager B1 adjuntos en la escena.	OK
Tareas	Entrega y retorno de T1 y T2; cuatro tareas completadas.	OK
Bill	Movimiento entre estaciones y acciones de tomar/trabajar/entregar.	OK
Percepción	16 sensores activos y rayos visualizados.	OK
Seguridad	Modo AVOIDING observado y señal de riesgo activa.	OK
Estimación	Kalman activo, mapa SLAM actualizado y error acotado.	OK
Planificación	SLAM_WAYPOINT observado.	OK
Batería	Tarea aceptada por batería y retorno a carga.	OK
Capturas CoppeliaSim	Vista general, entrega, trabajo de B1, rayos de sensores y replanificación Phase 2.	OK
Phase 2	Mapa desconocido, tarima móvil, bloque temporal, cuatro SLAM completados y recuperación del planificador registrada.	OK
Deck editable	Existe <code>slides/Actividad2_Pioneer_CoppeliaSim.pptx</code> junto con su PDF exportado.	OK

9.2 Timeline de la misión

La Fig. 9.1 muestra el timeline exportado con los estados de tarea y movimiento durante la misión.

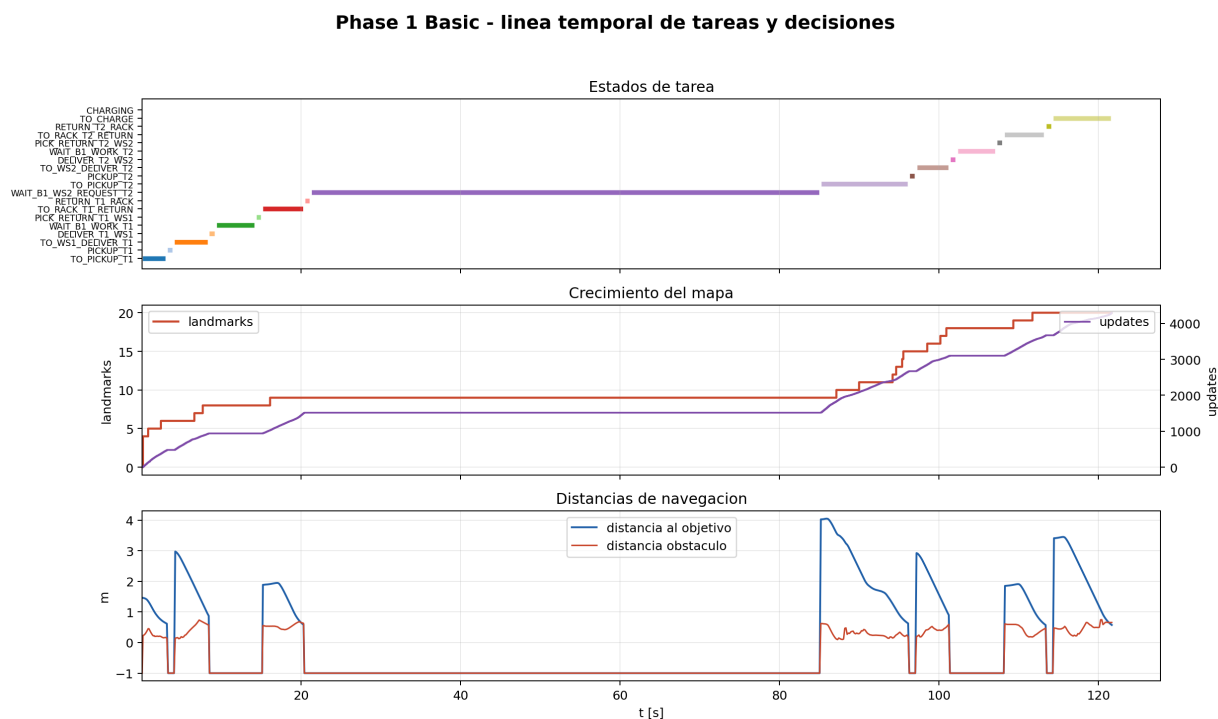


Figura 9.1: Timeline exportado de la simulación: estados de tarea, movimiento, planificador y acciones de Bill durante la misión.

La misión incluye esperas de proceso. R1 se detiene mientras Bill trabaja T1 o T2 y espera a que B1 llegue a WS2 antes de buscar T2. Esa espera aparece como WAIT_B1; sin ese estado, la escena presenta movimiento sin respetar la secuencia lógica de entrega.

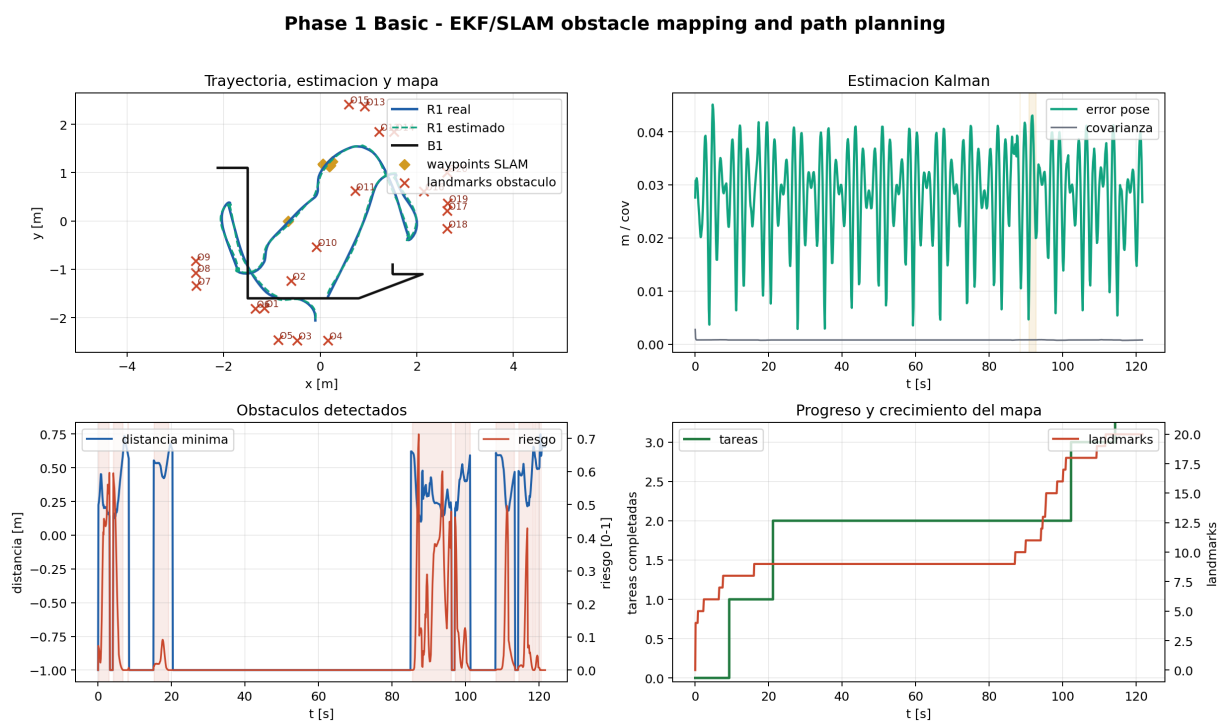


Figura 9.2: Panel de la ejecución base: pose, error, batería, obstáculos, planificador y estados principales.

El panel completo de la ejecución base queda recogido en la Fig. 9.2.

9.3 Incidencias corregidas

Durante la validación se corrigieron tres incidencias.

Incidencia 1 — Sincronización de tarea T2. R1 podía iniciar T2 antes de que Bill llegara a WS2, por lo que la tarea no representaba una entrega real. *Causa raíz:* la FSM no verificaba el estado de Bill al hacer la transición `RETURN_T1_RACK→T0_PICKUP_T2`; la transición ocurría en cuanto R1 completaba la tarea anterior, independientemente de la posición de Bill. Se corrigió dejando el estado `WAIT_B1_WS2_REQUEST_T2` bloqueado hasta leer `phase1B1Station=WS2`.

Incidencia 2 — Ciclos del planificador en Phase 2. Algunas celdas de grid persistentes quedaban demasiado cerca del objetivo y el planificador entraba en un ciclo de desvíos cortos. *Causa raíz:* las representaciones de grid acumulan evidencias pasadas que no se desvanecen; cuando un obstáculo ha desaparecido o el objetivo está dentro del radio de influencia de celdas marcadas, el planificador oscila indefinidamente. El parche exige detecciones persistentes, ignora celdas pegadas al objetivo y activa `RECOVERY_DIRECT` cuando no hay progreso. Esta fragilidad es un límite conocido de planificadores sobre grids de ocupación sin olvido temporal.

Incidencia 3 — Oscilación del scan matching tipo Hector. Con pocos impactos de sensor, varios candidatos de pose recibían puntajes parecidos y la corrección oscilaba. *Causa raíz:* el scan matching en entornos con poca textura (pasillos rectos) produce un paisaje de puntuación casi plano, en el que mínimas diferencias de ruido invertían el candidato ganador ciclo a ciclo. Es un modo de fallo conocido del ajuste de scan en geometrías poco informativas [11]. Se mitigó con ventana discreta pequeña, umbral mínimo de puntuación y ganancia parcial de corrección.

Conclusiones

La escena base comprueba el ciclo completo entre R1, Bill, T1 y T2: el robot entrega herramientas, espera el trabajo humano, recupera cada pieza, evita obstáculos y termina en carga. La misión incluye cola de tareas, sensores, estimación y retorno a estación.

Con los datos exportados, Kalman-landmark obtiene en Fase 2 el menor RMSE de posición (0,03185 m), la menor duración (124,80 s) y el menor consumo de batería entre los cuatro métodos, y se mantiene como candidato operativo para esta celda compacta. Las variantes inspiradas en GMapping, Hector y Cartographer se emplean para comparar tres representaciones de mapa: grid log-odds, ajuste de pose por scan-matching discreto y submapas con cierre de ciclo local. Las cuatro variantes usaron las mismas lecturas de proximidad y el mismo controlador PID para garantizar condiciones comparables. Cada modo se ejecutó en una única carrera de simulación ($n = 1$); la ordenación por métricas es indicativa y puede variar ante diferentes condiciones de inicialización o trayectorias de Bill.

El anticolisiones funciona como capa reactiva: cuando los sensores detectan algo cerca, el robot baja velocidad, modifica el giro y actualiza la señal de riesgo. La capa de punto intermedio SLAM filtra celdas de grid poco observadas, ignora obstáculos pegados al objetivo y penaliza desvíos fuera del área útil. En mapas de grid añadimos una recuperación por falta de progreso, con ruta directa durante una ventana corta y evitación reactiva siempre activa.

La comparación entre P, PI, PID, LQR y NMPC se conserva como análisis complementario. En el seguidor de Bill, PID dio el menor error final (0,0317 m), NMPC el menor error medio durante movimiento (0,0198 m) y LQR mantuvo buen error medio con menos variación que PID. En la misión T1 y T2, PI obtuvo el mejor puntaje ponderado de los CSV disponibles. LQR se aplicó como regulador discreto local y NMPC como control predictivo por disparo directo, resuelto con búsqueda local de comandos.

Limitaciones y trabajo futuro

La comparación de controladores se realizó con una única carrera por modo ($n = 1$) y sin análisis de estabilidad formal ante saturaciones, conmutación entre modos o cambios dinámicos de objetivo. La validación es experimental: las ejecuciones completan la misión sin divergencia observable, pero una prueba formal requeriría análisis de Lyapunov o estabilidad de modos conmutados. Las variantes de mapeo tampoco se evaluaron contra una referencia geométrica absoluta: el RMSE se mide respecto a la pose ruidosa del simulador, y las métricas de cobertura son proxies de actividad del mapa, no mediciones de IoU (*Intersection over Union*) contra un mapa de referencia.

Además de la bibliografía académica, se consultaron repositorios públicos para contrastar estructuras de implementación: `slam_gmapping`, `hector_slam`, Cartographer y PythonRobotics [16, 17, 18, 19]. Gemini y Claude se usaron como apoyo de consulta e ideación visual; el código final, las métricas, las figuras y la redacción técnica fueron revisados y adaptados manualmente [20].

Como trabajo futuro, el sistema podría conectarse a ROS 2 para ejecutar los paquetes originales de

slam_toolbox, Cartographer, Hector o GMapping, lo que permitiría comparar las implementaciones Lua contra sus homólogos completos. La evaluación de mapas debe incorporar referencia geométrica, IoU de ocupación, precisión y exhaustividad por obstáculo y varias ejecuciones por método para obtener significancia estadística.

Referencias

- [1] Universidad Internacional de Valencia, "Actividad 2: Programacion y control de un robot terrestre tipo pioneer en coppeliasim/luar," 2026. Material de la asignatura Sistemas Roboticos Moviles.
- [2] R. Siegwart, I. R. Nourbakhsh, and D. Scaramuzza, *Introduction to Autonomous Mobile Robots*. MIT Press, 2 ed., 2011.
- [3] Coppelias Robotics, "Coppeliasim user manual," 2026. Documentacion del simulador CoppeliaSim.
- [4] S. G. Tzafestas, *Introduction to Mobile Robot Control*. Elsevier, 2014.
- [5] MobileRobots Inc., *Pioneer 3 Operations Manual*, 2006. Manual de operación para plataformas Pioneer 3.
- [6] B. D. O. Anderson and J. B. Moore, *Optimal Control: Linear Quadratic Methods*. Prentice Hall, 1990.
- [7] J. B. Rawlings, D. Q. Mayne, and M. M. Diehl, *Model Predictive Control: Theory, Computation, and Design*. Nob Hill Publishing, 2 ed., 2017.
- [8] S. M. LaValle, *Planning Algorithms*. Cambridge University Press, 2006.
- [9] O. Khatib, "Real-time obstacle avoidance for manipulators and mobile robots," *The International Journal of Robotics Research*, vol. 5, no. 1, pp. 90–98, 1986.
- [10] V. Braitenberg, *Vehicles: Experiments in Synthetic Psychology*. MIT Press, 1984.
- [11] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. MIT Press, 2005.
- [12] R. E. Kalman, "A new approach to linear filtering and prediction problems," *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960.
- [13] G. Grisetti, C. Stachniss, and W. Burgard, "Improved techniques for grid mapping with rao-blackwellized particle filters," *IEEE Transactions on Robotics*, vol. 23, no. 1, pp. 34–46, 2007.
- [14] S. Kohlbrecher, O. von Stryk, J. Meyer, and U. Klingauf, "A flexible and scalable slam system with full 3d motion estimation," in *2011 IEEE International Symposium on Safety, Security, and Rescue Robotics*, pp. 155–160, 2011.
- [15] W. Hess, D. Kohler, H. Rapp, and D. Andor, "Real-time loop closure in 2d lidar slam," in *2016 IEEE International Conference on Robotics and Automation*, pp. 1271–1278, 2016.
- [16] ROS Perception, "slam_gmapping," 2018. Consultado en mayo de 2026. Repositorio público: https://github.com/ros-perception/slam_gmapping.
- [17] TU Darmstadt ROS Package Repository, "hector_slam," 2014. Consultado en mayo de 2026. Repositorio público: https://github.com/tu-darmstadt-ros-pkg/hector_slam.

-
- [18] Cartographer Project, "Cartographer," 2016. Consultado en mayo de 2026. Repositorio público: <https://github.com/cartographer-project/cartographer>.
- [19] A. Sakai, D. Ingram, J. Dinius, K. Chawla, A. Raffin, and A. Paques, "PythonRobotics," 2018. Consultado en mayo de 2026. Referencia de algoritmos de robótica móvil: <https://github.com/AtsushiSakai/PythonRobotics>.
- [20] Google, "Gemini," 2026. Herramienta usada como apoyo de consulta e ideación visual; no usada como fuente primaria de validación técnica.



Anexos

A.1 Archivos principales

Archivo	Función
coppeliasim/Sim_T2_Phase1_Basic.ttt	Escena final validada.
coppeliasim/Sim_T2_Phase1_Basic_validation.json	Resultado de validación automática.
coppeliasim/build_phase1_basic_scene.py	Generador de escena y validador.
coppeliasim/phase1_r1_task_controller.lua	Controlador principal de R1: tareas, navegación, SLAM, batería y telemetría.
coppeliasim/phase1_b1_walking_manager.lua	Movimiento y acciones visuales de B1 (Bill).
coppeliasim/capture_coppeliasim_screenshots.lua	Capturas 3D renderizadas desde CoppeliaSim para el informe.
coppeliasim/Sim_T2_Phase2_SLAM_Unknown.ttt	Escena con mapa desconocido, obstáculos no modelados y tarima móvil.
coppeliasim/build_phase2_unknown_slam_scene.py	Generador y validador de Phase 2.
coppeliasim/Actividad2_1_Basic_Follower.ttt	Escena básica de seguimiento de Bill.
coppeliasim/pioneer_pid_follower_controller.lua	Controlador del seguidor con P, PI, PID, LQR y NMPC por disparo directo.

Cuadro A.1: Escenas y controladores principales.

Archivo	Función
<code>coppeliiasim/run_phase1_slam_export.py</code>	Exportación CSV de la ejecución base.
<code>coppeliiasim/run_phase1_slam_comparison.py</code>	Comparación de algoritmos SLAM implementados en Lua.
<code>coppeliiasim/plot_phase1_slam_comparison.py</code>	Generación de gráficas de comparación SLAM.
<code>coppeliiasim/phase1_slam_logs/</code>	CSV, JSON, Markdown y figuras de la comparación SLAM.
<code>coppeliiasim/run_phase2_slam_unknown_comparison.py</code>	Exportación comparativa Phase 2 para los cuatro modos SLAM.
<code>coppeliiasim/plot_phase2_slam_unknown_comparison.py</code>	Figuras y métricas de reconstrucción Phase 2.
<code>coppeliiasim/phase2_slam_logs/</code>	CSV, JSON, Markdown y figuras de Phase 2.
<code>coppeliiasim/phase1_controller_logs/</code>	Registros CSV y gráficas de comparación de controladores.
<code>coppeliiasim/follower_pid_logs/</code>	CSV y gráficas de seguimiento de Bill.

Cuadro A.2: Exportadores, registros y escena complementaria.

A.2 Parámetros relevantes

Parámetro	Valor	Comentario
Radio de rueda	0,0975 m	Conversión de velocidad lineal a angular de motor.
Distancia entre ruedas	0,331 m	Modelo diferencial del Pioneer 3DX.
Velocidad máxima (vMax)	0,62 m/s	Saturación de avance (controlador de misión).
Velocidad angular máxima (wMax)	1,42 rad/s	Saturación de giro (controlador de misión).
Rango de evitación	0,54 m	Umbral de influencia reactiva de obstáculos.
Parada crítica	0,10 m	Distancia frontal mínima antes de detener el avance.
Batería inicial	96 %	Valor usado en validaciones.
Umbral batería baja	56 %	Limita velocidad si se alcanza.
Umbral batería crítica	26 %	Modo conservador.
Resolución de grid	0,18 m	Usada por GMapping, Hector y Cartographer en grid.
Margen del planificador	0,40 m	Separación mínima deseada respecto al mapa.
Desplazamiento lateral	0,52 m	Distancia lateral del punto intermedio local.
Retardo de recuperación	6,0 s	Activa RECOVERY_DIRECT si no hay progreso hacia el objetivo.
Duración de recuperación	3,5 s	Ventana en la que se suspende el mapa para planificar, sin apagar la evitación local.

A.3 Fragmentos Lua propios

Listing A.1: Conversión de velocidad diferencial a ruedas.

```
function DifferentialDrive:setVelocity(v, w)
    local left = (v - 0.5 * self.cfg.trackWidth * w) / self.cfg.wheelRadius
    local right = (v + 0.5 * self.cfg.trackWidth * w) / self.cfg.wheelRadius
    sim.setJointTargetVelocity(self.handles.leftMotor, left)
    sim.setJointTargetVelocity(self.handles.rightMotor, right)
end
```

Listing A.2: Entrega, espera de trabajo humano y retorno de herramienta.

```
local function deliverToolToBill(tool, workSurface, completedCount, nextSpec,
    nextState, afterDeliver)
    stopRobot('MANIPULATING')
    if delayElapsed(cfg.manipulationDelay) then
        carryingTool = 0
        placeToolAt(tool, workSurface, 0.43)
        completedTaskCount = math.max(completedTaskCount, completedCount)
        currentTaskSpec = nextSpec
        setShapeColorSafe(tool, ToolColor.onTable)
        if afterDeliver then afterDeliver() end
        setTaskState(nextState)
    end
end
```

```

local function waitForBillWork(tool, workSurface, dropPoint, workDelay,
    nextTaskId, nextSpec, nextState)
    stopRobot('WAIT_B1')
    placeToolAt(tool, workSurface, 0.43)
    if delayElapsed(workDelay) then
        currentTaskId = nextTaskId
        currentTaskSpec = nextSpec
        placeToolAt(tool, dropPoint, 0.36)
        setTaskState(nextState)
    end
end
end

```

Listing A.3: Animación de Bill trabajando con brazos hacia la mesa.

```

elseif self.actionContains(action, 'WORKING') then
    pose.leftShoulder = 0.62 + 0.06 * pulse
    pose.rightShoulder = 0.60 - 0.06 * pulse
    pose.leftElbow = 0.28 + 0.05 * math.abs(tap)
    pose.rightElbow = 0.26 + 0.05 * math.abs(tap)
    pose.leftKnee = 0.07 + 0.03 * math.abs(pulse)
    pose.rightKnee = pose.leftKnee

```

A.4 Comparación de control

Las métricas complementarias de control se ilustran en la Fig. A.1.

Modo	Dur. [s]	Mov. [s]	Ruta [m]	Bat. [%]	Riesgo max.	Puntaje
P	115,20	37,70	12,843	19,804	0,410	0,573
PI	110,30	31,20	12,867	19,516	0,487	0,274
PID	110,85	31,55	12,856	19,784	0,469	0,567
LQR	114,15	36,20	12,796	19,505	0,401	0,436
NMPC	112,30	33,05	12,788	19,316	0,444	0,326

Cuadro A.3: Resultados comparativos de los cinco controladores de seguimiento en Fase 1.

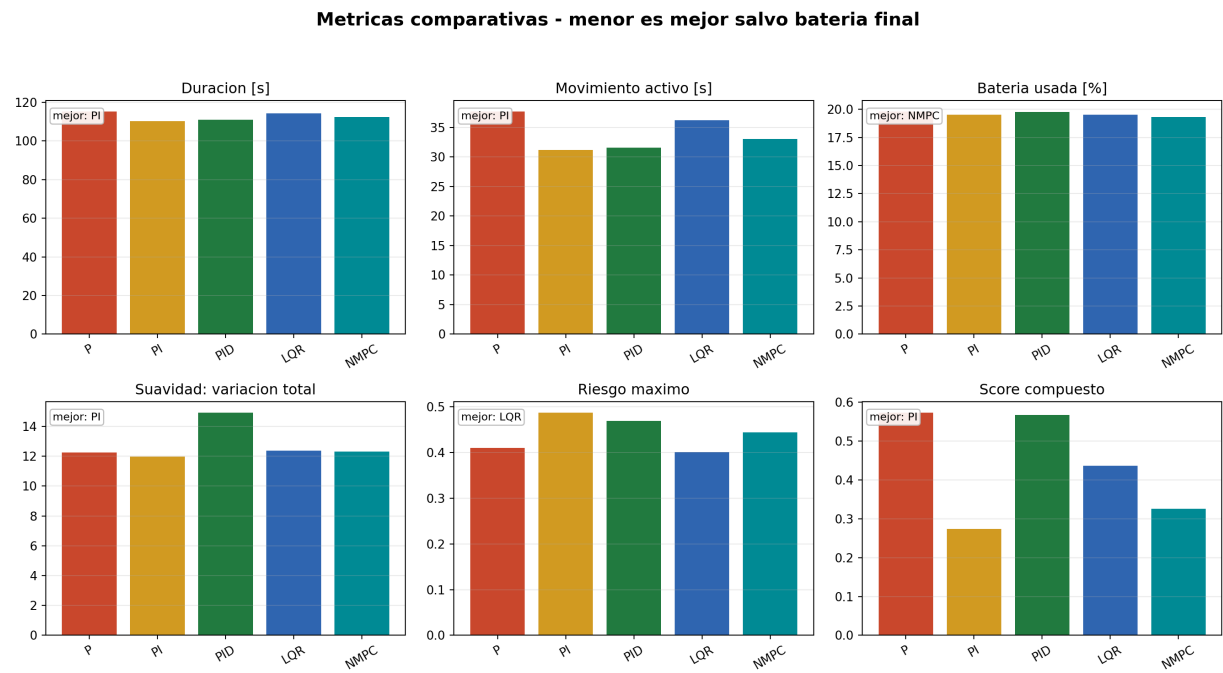


Figura A.1: Métricas complementarias de control: tiempos, trayectoria, suavidad, batería, riesgo y error de pose.

A.5 Parámetros de Cartographer

Los parámetros del módulo Cartographer-submap usados en Fase 1 y Fase 2 son (Tabla A.4):

Parámetro	Valor
cartographer.submap_min_age_s	22 s
cartographer.submap_max_distance_m	0,45 m
cartographer.loop_closure_score	0,35

Cuadro A.4: Parámetros de Cartographer-submap. Idénticos en Fase 1 y Fase 2.

A.6 Notas de alcance

- Las variantes GMapping, Hector y Cartographer son implementaciones Lua inspiradas en esos métodos y calculadas con los datos de los sensores de la escena.
- El sensor se representa con los 16 ultrasonidos del Pioneer visualizados como rayos. Las figuras de reconstrucción SLAM corresponden a esos rayos de proximidad y al arreglo ultrasónico simulado.
- phase2MappingEvidencePct mide actividad de mapeo. La IoU y la cobertura geométrica contra una referencia real quedan fuera de esa señal.
- La planificación local por punto intermedio resuelve bloqueos de una celda pequeña sin introducir una capa global adicional.
- LQR y NMPC se usan como comparaciones acotadas; el análisis central usa métodos clásicos de

seguimiento, evitación y estimación.